INE EJPT STUDY GUIDE

eJPT Study Guide: Module 1.1 Information Gathering

Assessment Methodologies. Passive and active reconnaissance, DNS, subdomain discovery, host and port discovery, web footprinting.

Authorized labs and legal practice only.

Contents

1. Module at a Glance 4
The 10 things to remember most 4
Skill checklist 5

2. Big Picture Map 5

3. Concept-by-Concept Guide 6
3.1 What Is Information Gathering 6
3.2 Target Scoping 7
3.3 Passive vs Active Reconnaissance 8
3.4 What Information We Collect 9
3.5 Recon Flow, Four-Step Strategy and Common Mistakes 10
3.6 DNS Lookups with host 12
3.7 robots.txt and sitemap.xml 14
3.8 Technology Profiling with Browser Extensions (BuiltWith and Wappalyzer) 16
3.9 WhatWeb 17
3.10 WHOIS 19
3.11 Netcraft Site Report (including SPF, DMARC and Web Trackers) 21
3.12 DNSDumpster 23
3.13 HTTrack Website Copier 24
3.14 dnsrecon 26
3.15 wafw00f (WAF Detection) 28
3.16 Sublist3r (Subdomain Enumeration) 30
3.17 Google Dorking and the Google Hacking Database 32
3.18 theHarvester 34
3.19 DNS and DNS Record Types 37
3.20 DNS Interrogation and dig 38
3.21 DNS Zone Transfer (AXFR) with dig 41
3.22 dnsenum 43
3.23 fierce 45
3.24 Host Discovery with Nmap Ping Sweep (nmap -sn) 47
3.25 netdiscover 49
3.26 Nmap Without Ping: -Pn 51
3.27 Fast Scan: -F 53
3.28 Service Version Detection: -sV 54
3.29 OS Detection, Default Scripts and Verbose: -O -sC -v 56
3.30 Reading Page Source: the HTML Generator Meta Tag 58
3.31 Directory Brute-Forcing with dirb 60
3.32 Backup Files in the Web Root 62
3.33 CTF Walkthrough: Information Gathering CTF 1 65

4. Chained Scenarios 67
Scenario A: External domain, passive to active 67
Scenario B: Internal lab network to a profiled web server 67
Scenario C: Web target flag hunt (CTF style) 68

5. Decision Guide and Exam Strategy 69
If I see X, I try Y 69
Enumeration checklist for this module 69
Common traps 70
How to take notes in the lab 70
Turning findings into answers 71

6. Practice 72
6.1 Lab-style tasks 72
6.2 Exam-style questions 77
6.3 Answer key 78

7. Recap Card and Flashcards 80
7.1 Recap Card 80
7.2 Flashcards 80

8. Glossary 82

9. Coverage Report 84
9.1 Page-to-section map 84
9.2 Source limits and [UNCLEAR] items 85
9.3 Items to double check 85
9.4 Coverage confirmation 86

1. Module at a Glance

Module name: 1.1 Information Gathering (INE eJPT, Assessment Methodologies).

Exam domains this module feeds:

- Assessment Methodologies (25 percent): the main target. Scoping, passive vs active recon, DNS, OSINT, host discovery, port scanning.
- Host and Network Auditing (25 percent): Nmap host, port, service and OS detection.
- Web Application Penetration Testing (15 percent): robots.txt, technology fingerprinting, directory brute-forcing, backup files, WAF detection.
- Host and Network Penetration Testing (35 percent): every attack starts from the target list you build here.

What the exam expects from this module: You must find hosts, open ports, service versions, operating systems, subdomains, DNS records and hidden web content in a live lab, then answer questions with the exact values you found. The 2026 exam update added target scoping, passive vs active reconnaissance, and file and directory brute-forcing. All three are covered here.

> **ADDED CONTEXT:** The exam has 45 questions over 48 hours, open book, in a live lab. Pass mark is reported as 70 percent overall plus a minimum score per domain. Your notes do not contain these facts. They agree with the exam facts you supplied.

The 10 things to remember most

1. Scope first. Never touch anything that is not written in the scope.
2. Passive recon never sends packets to the target. Active recon does.
3. Do passive first, then active. Write down every result.
4. `host`, `dig`, `whois` and `dnsrecon` read DNS and registration data. Learn the record types A, AAAA, NS, MX, CNAME, TXT, HINFO, SOA, SRV, PTR.
5. A DNS zone transfer (`dig axfr @server domain`) can leak the whole zone. It works only when the server is misconfigured.
6. `nmap -sn` finds live hosts. `nmap -Pn` skips the ping check when ICMP is blocked.
7. `nmap -sV` finds service versions. `-O` guesses the OS. `-sC` runs default scripts. `-F` scans the top 100 ports.
8. Always check `/robots.txt`, page source (meta generator tag), hidden directories (dirb) and backup files (`wp-config.php.bak`).
9. Tool output can be wrong or incomplete. Verify it by hand (curl, browser).
10. Save every command and result. Do not repeat work, and use your notes to answer questions.

Skill checklist

- I can define a scope and say whether a host or domain is in or out of scope.
- I can explain passive vs active recon and give three examples of each.
- I can resolve a domain to IPs, mail servers and name servers with `host` and `dig`.
- I can read a `whois` record and name the registrar, creation date and name servers.
- I can identify web technologies with WhatWeb, Wappalyzer, BuiltWith and Netcraft.
- I can detect a WAF with `wafw00f`.
- I can enumerate subdomains with Sublist3r, dnsrecon, dnsenum, fierce and DNSDumpster.
- I can collect emails and hostnames with theHarvester and Google dorks.
- I can test a name server for a zone transfer and read the result.
- I can find live hosts with `nmap -sn` and `netdiscover`.
- I can scan ports, versions, OS and default scripts with Nmap.
- I can find flags in robots.txt, page source, hidden directories, backup files and website mirrors.

> **REMEMBER:** Use these techniques only on systems you own or have written permission to test. This includes the INE labs and the eJPT exam lab.

2. Big Picture Map

Information gathering is the first real step after scoping. Everything later depends on it.

```
[Scoping] -> [RECON passive then active] -> [Scan and enumerate]
                 ^ this module                        |
                                                      v
[Reporting] <- [Pivoting] <- [Priv-esc] <- [Exploitation] <- [Vuln assess]
```

Where each part of this module fits:

Pentest phase	What this module gives you
Scoping	Domain, IP and application targets. In and out of scope
Passive recon	host, whois, Netcraft, DNSDumpster, dorks, theHarvester
Active recon	dnsrecon, dnsenum, dig, fierce, nmap, netdiscover
Scanning and enumeration	nmap -sV -O -sC, dirb, WhatWeb, wafw00f
Vulnerability assessment	Versions, backup files, exposed directories
Exploitation	The service list you will attack in later modules
Pivoting and reporting	Internal IPs seen in DNS data; your saved notes

3. Concept-by-Concept Guide

3.1 What Is Information Gathering

In one line: Information gathering (also called reconnaissance or recon) is collecting facts about a target before you attack it.

Theory in simple words: Think of a burglar who studies a building before doing anything: the doors, the guards, the opening hours. A penetration tester does the same with a company, a website or a network. The more facts you hold, the easier the later steps are.

There are two types:

- **Passive information gathering:** you collect facts without directly engaging the target. Example: reading public records.
- **Active information gathering:** you engage the target system directly. Example: scanning its ports. This needs authorization.

How it works:

1. Receive the target and the scope (3.2).
2. Collect public data first (passive).
3. Contact the target systems (active) only with authorization.
4. Record everything in organized notes.
5. Hand the results to the enumeration and exploitation phases.

Worked example: The client gives you `example.com`. You read its DNS and WHOIS data first (passive). Then you scan the IPs you found (active). You write down: 3 IPs, 2 mail servers, ports 22, 80 and 443 open.

Lab question angle: "Which type of recon is `whois`?" -> passive. "Which type is an Nmap scan?" -> active.

Why it works and how defenders stop it: Most information is public by design (DNS, websites). Defenders limit it by using WHOIS privacy, removing version banners and monitoring for scans.

Memory aid: Passive = Public data. Active = Asks the target (packets sent).

> **EXAM TIP:** If a question says "without sending traffic to the target", the answer is passive reconnaissance.

3.2 Target Scoping

In one line: Scoping defines exactly which systems, networks and applications you are allowed to test.

Theory in simple words: The scope is your permission slip. You test only what is on it. The key question it answers for information gathering is: "What am I allowed to collect information about?" Anything you can find but is not listed is not yours to test.

Analogy: a locksmith hired to open one door does not open the neighbor's door, even if it is easy.

Common target types:

Type	Examples	Typical use
Domain-based	example.com, testsite.local	External tests
IP-based	192.168.1.10 or 192.168.1.0/24	Internal and lab networks
Application-based	One web app, login portal, API endpoint	Web app tests

- **Domain-based:** may include the main domain and subdomains such as `mail.example.com` and `admin.example.com`.
- **IP-based:** a single IP such as `192.168.1.10`, or a network range such as `192.168.1.0/24` (CIDR notation: `/24` means 256 addresses, `192.168.1.0` to `192.168.1.255`).
- **Application-based:** recon focuses on that application only, not the entire server.

In-scope assets are those you may collect information from, scan and enumerate.

Out-of-scope assets you must not interact with. Examples from the notes: third-party services, external domains not listed in scope, systems owned by another organization.

How it works:

1. Read the rules of engagement and the scope list.
2. Write the allowed domains, IPs and applications in your notes.
3. Check every new host or domain you discover against that list.
4. If it is not listed, do not scan it. Report it instead.

Worked example:

```
Scope list:   example.com, *.example.com, 10.10.10.0/24
Found:        mail.example.com        -> in scope (subdomain)
Found:        cdn.thirdparty.net      -> out of scope (third party)
Found:        10.10.10.5              -> in scope (inside the /24)
Found:        10.10.20.9              -> out of scope (different range)
```

> **ADDED CONTEXT:** A site hosted on a cloud service or CDN (a network of caching servers) shares IPs with other customers. Scanning that IP can hit systems you are not allowed to test. Scope by hostname, not only by IP.

Lab question angle: In the exam lab the scope is a set of IPs or a domain. Questions can ask which discovered host falls outside the scope, or which range to scan first. Compare each IP to the scope text with CIDR math: `10.10.10.0/24` covers `10.10.10.0` to `10.10.10.255` only.

Why it works and how defenders stop it: Scoping is a legal and safety control, not an attack. It protects both the client and the tester. In reports, always state the scope used.

Memory aid: If it is not on the list, it is not on the test.

> **TRAP:** A subdomain you discover is not automatically in scope. Check whether the scope says `*.example.com` or only `example.com`.

3.3 Passive vs Active Reconnaissance

In one line: Passive recon reads public data without touching the target. Active recon sends traffic to the target.

Theory in simple words: Passive is like reading about a house in the town records. Active is like walking up and knocking on its doors.

Feature	Passive	Active
Direct connection to target	No	Yes
Risk of detection	Low	Higher
When	Usually first	After passive
Needs authorization	Public data	Yes

Passive data examples (notes): domain registration information, DNS records, public website content, search engine results, publicly available email addresses.

Active data examples (notes): live hosts, open ports, running services, network responses.

> **ADDED CONTEXT:** The notes slide for active recon is titled "Examples of passive recon data" by mistake. The listed items (live hosts, open ports, services) are active data. This guide uses the corrected title.
>
> **ADDED CONTEXT:** The line between the two is not always sharp. A `host` or `dig` query goes to a public DNS resolver, often not the target's own server. This is usually treated as passive. A query sent straight to the target's name server, or a zone transfer, is active. Browsing the target website normally is low-risk but is still contact with the target.

Which tool is which in this module:

Technique	Type	Section
whois, Netcraft, DNSDumpster	Passive	3.10, 3.11, 3.12
Google dorks, theHarvester	Passive	3.17, 3.18
Sublist3r (search engines and OSINT)	Passive	3.16
dnsrecon, dnsenum, fierce, zone transfer	Active (query target DNS)	3.14 to 3.23
nmap, netdiscover, dirb, wafw00f, WhatWeb	Active	3.24 to 3.32, 3.9, 3.15

Worked example: For `example.com` you run `whois`, then check Netcraft. No packet reaches the company. Then you run `nmap -sn` on the authorized range. That sends ICMP and ARP packets, so it is active.

Lab question angle: Multiple choice: "Which of these is a passive recon technique?" Pick the one that uses public data (WHOIS, search engine, Netcraft) and not a scan.

Why it works and how defenders stop it: Passive data cannot be blocked because it lives on third-party systems. Defenders reduce exposure by trimming what they publish. Active recon shows up in firewall and IDS logs.

Memory aid: Passive = Public. Active = Alarms can ring.

3.4 What Information We Collect

In one line: The goal of recon is to build a map of the target.

Theory in simple words: You are drawing a map before a trip. The map lists the roads (IPs and ports), the buildings (hosts and services) and the signs on them (versions and technologies).

Targets of collection (page 2 of the notes):

- Domains and subdomains
- IP addresses
- Hosting infrastructure (who hosts the server)
- Technologies and services
- Open ports
- Publicly exposed information

Specific items to look for (page 4 of the notes):

- IP addresses
- Directories hidden from search engines
- Names
- Email addresses

- Phone numbers
- Physical addresses
- Web technologies being used

How it works: Each item maps to a method used later in this module:

Item	Where to find it	Section
IP addresses	host and dig queries	3.6, 3.20
Hidden directories	robots.txt, dirb	3.7, 3.31
Names, emails	theHarvester, WHOIS	3.18, 3.10
Phones, addresses	WHOIS, website, Netcraft	3.10, 3.11
Web technologies	WhatWeb, Wappalyzer	3.8, 3.9
Subdomains	Sublist3r, dnsrecon, fierce	3.16, 3.14, 3.23
Open ports	nmap	3.26, 3.27

Worked example: For `example.com` your notes table might read: IP 203.0.113.10; mail server mail.example.com; hosting provider (from Netcraft); web server Apache; CMS WordPress.

Lab question angle: "What is the IP address of mail.target.local?" or "Which web server software does the site run?" You answer from your saved records.

Why it works and how defenders stop it: Organizations leak small facts across many places. Defenders audit what is public (email formats, server banners, old files).

Memory aid: IP, Names, Tech, Ports, Files. Collect them all, note them all.

3.5 Recon Flow, Four-Step Strategy and Common Mistakes

In one line: A fixed order of steps keeps recon complete and avoids repeated work.

Theory in simple words: A checklist stops you from forgetting steps. The notes give a four-step strategy to use on every engagement.

The recon mapping flow (rebuilt from the garbled diagram on page 3):

```
              [Define Target Scope]
                       |
                       v
           [Passive Reconnaissance]
              |                |
              v                v
      [Domains/Subdomains]  [DNS Records]
      [Whois Data]          [Website Footprinting]
      [OSINT (search engines emails breach awareness)]
      [Technology Fingerprinting]
              |
              v
           [Active Reconnaissance]
      [Host Discovery] -> [Port Scanning]
      [Basic Service Identification]
      [DNS Zone Transfer Testing]
              |
              v
       [Organize Findings]
              |
              v
 [Proceed to Enumeration and Exploitation]
```

The four steps:

1. **Define the target.** Identify the domain, IP address or network range. Confirm what is in scope.
2. **Perform passive recon.** Gather public information, identify possible attack surfaces, build an initial understanding of the target.
3. **Perform active recon.** Discover live hosts, identify open ports, detect exposed services.
4. **Document and organize your findings.** Record domains, IPs and ports. Prepare information for enumeration. Avoid repeating work later.

Common beginner mistakes (notes):

- Starting scans without defining scope.
- Skipping passive reconnaissance.
- Scanning everything instead of relevant targets.
- Not documenting results.
- Trusting tool output without verification.

Key takeaways (notes):

- Target scoping defines what you are allowed to test.
- Passive recon comes before active recon.
- Recon is about collecting meaningful data.
- A structured approach improves results and efficiency.

Worked example (what a good notes file looks like):

```
TARGET: example.com   SCOPE: example.com, 10.10.10.0/24
PASSIVE: whois -> registrar NameCheap; NS ns1.example.com
PASSIVE: Netcraft -> Apache, no DMARC
ACTIVE : nmap -sn -> 10.10.10.5, 10.10.10.9 up
ACTIVE : nmap -sV 10.10.10.5 -> 22 ssh, 80 http (Apache 2.4.x)
TODO   : dirb on 10.10.10.5, zone transfer on ns1
```

Lab question angle: Ordering questions: "What is the first step of recon?" -> define the target and scope. "What do you do after active scanning?" -> document and prepare for enumeration.

Why it works and how defenders stop it: Method beats luck. In reports, the recon section proves you followed a repeatable process.

Memory aid: S-P-A-D: Scope, Passive, Active, Document.

> **EXAM TIP:** The lab has hidden internal subnets. Your notes are the only way to remember an IP found on a host that you will need again for pivoting.

3.6 DNS Lookups with host

In one line: `host` is a small command that turns a domain name into IP addresses and shows its mail servers.

Theory in simple words: DNS (Domain Name System) is the internet phone book. It maps names like `hackersploit.org` to IP addresses. `host` asks that phone book a question and prints a short answer. Full DNS theory is in 3.19.

How it works:

1. You give `host` a domain.
2. It sends a DNS query to your configured resolver (a DNS server that answers for you).
3. It prints the A (IPv4), AAAA (IPv6) and MX (mail) records.

Hands-on:

`host` is a DNS lookup tool installed by default on Kali (package `bind9-host`). We use it for quick, low-noise lookups.

Purpose: find the IP addresses and mail servers of a domain.

```
host hackersploit.org
```

Part	Meaning
host	The lookup program
hackersploit.org	The domain to look up

Output from your notes:

```
hackersploit.org has address 172.67.202.99
hackersploit.org has address 104.21.44.180
hackersploit.org has IPv6 address 2606:4700:3031::6815:2cb4
hackersploit.org has IPv6 address 2606:4700:3036::ac43:ca63
hackersploit.org mail is handled by 0 _dc-mx.2c2a3526b376.hackersploit.org.
```

How to read it:

- `has address` lines are A records (IPv4). Two IPs means the name points to two servers.
- `has IPv6 address` lines are AAAA records.
- `mail is handled by 0 <name>` is an MX record. The number is the priority. A lower number is tried first.

Second example from your notes:

```
host aniketpandey.website
```

```
aniketpandey.website has address 216.198.79.1
aniketpandey.website mail is handled by 10 mx00.ionos.com.
aniketpandey.website mail is handled by 10 mx01.ionos.com.
```

Two mail servers with the same priority (10) share the load. The mail provider here is IONOS, which tells you who hosts the email.

Useful variations:

```
host -t ns hackersploit.org
host -t mx hackersploit.org
host -t txt hackersploit.org
host 10.10.10.5
```

Part	Meaning
-t ns	Ask for NS (name server) records
-t mx	Ask for MX (mail) records
-t txt	Ask for TXT (text) records, such as SPF
10.10.10.5	An IP: does a reverse lookup (PTR record)

Common mistakes and fixes:

- Typing a URL (`http://site`) instead of a domain. Use only the name.
- Reading "mail is handled by" as the web server. It is the mail server.

- Two IPs on one name often mean a CDN or load balancer, not two separate companies' servers.

Worked example: Goal: find where `example.com` sends mail. Command: `host -t mx example.com`. You see `10 mail.example.com`. Next, `host mail.example.com` gives `10.10.10.9`. Write down: mail server 10.10.10.9. Decide: check if this IP is in scope (3.2).

Lab question angle: "What is the IPv4 address of the domain?" or "Which mail server handles mail?" Run `host` and read the A or MX line.

Why it works and how defenders stop it: DNS must be public for the site to work. Defenders cannot hide it, but they can avoid putting internal names in public DNS.

Memory aid: `host` = quick Who-is-this-IP. A = address, MX = mail.

3.7 robots.txt and sitemap.xml

In one line: `robots.txt` is a public file that tells search engines which paths not to crawl. It often shows hidden paths.

Theory in simple words: Think of a "staff only" sign on a door. It asks visitors to stay out, but it does not lock the door. And it also tells everyone where the door is. Your notes say: "robots.txt and sitemap.xml files are important".

Two files to check on every website:

- `/robots.txt` lists `Disallow` (do not crawl) and `Allow` rules for crawlers.
- `/sitemap.xml` lists the pages the owner wants indexed. It can show pages that are not linked anywhere.

How it works:

1. A search engine bot requests `/robots.txt` first.
2. It reads rules per `User-agent` (the bot name; `*` means all bots).
3. `Disallow` lines are paths the owner does not want crawled.
4. You, as the tester, read the same file and test those paths by hand.

Hands-on:

The notes show the file in a browser at `https://hackersploit.org/robots.txt`:

```
User-agent: *
Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php

Disallow: /wp-content/uploads/wpo-plugins-tables-list.json
```

How to read it:

- `User-agent: *` -> the rules apply to all crawlers.

- `Disallow: /wp-admin/` -> the WordPress admin area exists. This confirms the site runs WordPress.
- `Allow: /wp-admin/admin-ajax.php` -> an exception inside the disallowed folder.
- The last line shows a plugin file path that the owner wanted kept out of search.

Tool: curl (first introduced here). `curl` is a command-line program that sends web requests and prints the response. It is on Kali by default. We use it to read files and headers without a browser, and it is easy to script.

Purpose (CTF Flag 1): read robots.txt from the terminal.

```
curl http://target.ine.local/robots.txt
```

Part	Meaning
curl	The HTTP client
http://target.ine.local	The target web server (lab name)
/robots.txt	The file path requested

Illustrative output (the value is fake):

```
User-agent: *
Disallow: /admin/
Disallow: /backup/
# FL@G1{sample-flag-value}
```

On the lab target, your notes say "the flag is embedded directly inside the file content". Read every line, including comments that begin with `#`.

Useful variations:

```
curl -s http://target.ine.local/robots.txt
curl -s http://target.ine.local/sitemap.xml
curl -I http://target.ine.local/robots.txt
curl -s http://target.ine.local/sitemap.xml | grep -o "<loc>[^<]*"
```

Option	Meaning
-s	Silent: hides the progress meter
-I	Headers only (status code, server)
grep -o	Print only the matching part
`<loc>[^<]*`	Matches each URL tag in a sitemap

Common mistakes and fixes:

- Forgetting `http://`. Use the full URL.
- Getting a 404: not every site has the file. Try a browser too and check for redirects with `curl -L`.
- Assuming Disallow paths are protected. They are only a request. Visit them.

Worked example: Goal: find hidden areas on `10.10.10.5`. Run `curl http://10.10.10.5/robots.txt`. You see `Disallow: /backup/`. Open `http://10.10.10.5/backup/` with curl. You see a file list. Write down: /backup/ exists, files inside.

Lab question angle: "Which directory is disallowed in robots.txt?" or "Find the flag in robots.txt." Read the file in full, including comments.

Why it works and how defenders stop it: The file is public by design. Defenders should not list sensitive paths in it. Protect them with authentication instead.

Memory aid: robots.txt = a map of what the owner wants hidden.

> **EXAM TIP:** Check `/robots.txt` and `/sitemap.xml` in the first minute on every web server. It costs one command.

3.8 Technology Profiling with Browser Extensions (BuiltWith and Wappalyzer)

In one line: These two Firefox add-ons show which technologies a website uses, with one click.

Theory in simple words: A website is built from parts: a web server, a language, a CMS (Content Management System such as WordPress), JavaScript libraries, analytics and a CDN. The extensions read headers, cookies and page code to guess each part. Knowing the parts tells you which known vulnerabilities to look for later.

Tools from your notes:

- **BuiltWith** (by garazy): a web site profiler. It returns a list of the technologies it finds on the page.
- **Wappalyzer** (by Wappalyzer): identifies technologies on websites. The panel groups them by category.

How it works:

1. Install the add-on from the Mozilla Add-ons page in Firefox.
2. Open the target site.
3. Click the extension icon.
4. Read the list: web server, CMS, languages, frameworks, JS libraries, analytics, CDN.
5. Write down names and versions.

Hands-on: No terminal command. Note the findings in your notes file.

Example of what a panel shows (illustrative):

```
Web servers:   Apache HTTP Server 2.4.x
CMS:           WordPress 6.x
Languages:     PHP
JavaScript:    jQuery
CDN:           Cloudflare
```

Common mistakes and fixes:

- Trusting a version number without checking. Confirm with `curl -I`, WhatWeb (3.9) or page source (3.30).
- Expecting every site to show everything. Hidden or custom setups show less.

> **ADDED CONTEXT:** The eJPT lab network may have no internet access. Wappalyzer reads the page locally in the browser, so it can still work. BuiltWith may need an online lookup, so it may fail (verify in your lab). WhatWeb (3.9) always works from the terminal.

Worked example: Goal: learn the CMS of `http://10.10.10.5`. Open the site, click Wappalyzer, see WordPress and PHP. Decision: later use WPScan and look at `/wp-login.php`. Write down: WordPress, PHP.

Lab question angle: "Which CMS does the site use?" or "Which web server software?" Open the page and read the panel, then confirm with WhatWeb.

Why it works and how defenders stop it: Sites send identifying headers and file names. Defenders can remove version banners and generator tags, which only slows an attacker.

Memory aid: Wappalyzer = what apps are on this page.

3.9 WhatWeb

In one line: WhatWeb is a terminal scanner that identifies web technologies on a site.

Theory in simple words: It does the work of Wappalyzer from the command line. It requests the page and runs many small plugins that match patterns in headers and HTML (server name, CMS, cookies, titles).

How it works:

1. You give WhatWeb a URL, hostname, IP, file of targets or range.
2. It sends HTTP requests according to the aggression level.
3. Plugins match patterns and print one line per target.

Hands-on:

WhatWeb is on Kali by default (`whatweb`). Your notes show version 0.5.2 from Andrew Horton (urbanadventurer) and Brendan Coles (bcoles). It is active recon, because it sends requests to the target.

Usage line from the notes:

```
Usage: whatweb [options] <URLs>
```

Options shown in your notes:

Option	Meaning
TARGETs	URLs, hostnames, IPs, files or ranges (CIDR, 10.0.0-5)
--input-file=FILE, -i	Read targets from a file
--aggression, -a=LEVEL	Set the aggression level (default 1)
--list-plugins, -l	List all plugins
--info-plugins, -I=[SEARCH]	Plugin details, optional keyword search
--verbose, -v	Verbose output with plugin descriptions
-h or --help	Full help (the screen shows the short help)

Aggression levels in the notes:

- **1 (Stealthy):** one HTTP request per target; also follows redirects.
- **3 (Aggressive):** if a level 1 plugin matches, WhatWeb makes extra requests to learn more.

> **ADDED CONTEXT:** A level 4 (Heavy) exists in the full help (verify). Higher levels send more requests and are noisier.

Purpose: identify the technologies of `hackersploit.org` (command from your notes).

```
whatweb hackersploit.org
```

Part	Meaning
whatweb	The program
hackersploit.org	The target host (level 1 by default)

Illustrative output (the notes do not show the result; wrapped for print):

```
http://hackersploit.org [301 Moved Permanently] Country[UNITED STATES],
  HTTPServer[cloudflare], IP[104.21.44.180], RedirectLocation[https://
  hackersploit.org/]
https://hackersploit.org/ [200 OK] Cloudflare, HTML5, IP[104.21.44.180],
  MetaGenerator[WordPress 6.x], Title[HackerSploit], WordPress
```

How to read it: each line is one request. The first shows a redirect from HTTP to HTTPS (level 1 follows redirects). The second lists plugins that matched: CDN, HTML5, generator tag, the CMS.

Useful variations:

```
whatweb -a 3 -v http://10.10.10.5
whatweb -i targets.txt
whatweb 10.10.10.0/24
whatweb -l
```

Part	Meaning
-a 3	Aggressive mode: more requests when something matches
-v	Show plugin descriptions and more detail
-i targets.txt	Read one target per line from the file
10.10.10.0/24	Scan every address in the range
-l	List all available plugins

Common mistakes and fixes:

- Running level 3 against a live production site without permission. Stay at level 1 unless you are authorized.
- Giving a bare IP when the site needs a hostname. Add the name to `/etc/hosts` if the lab says so.

Worked example: Goal: check what runs on three lab hosts. Put `10.10.10.5`, `10.10.10.9` and `10.10.10.12` in `targets.txt`. Run `whatweb -i targets.txt`. You see Apache on one, IIS on another and WordPress on the third. Note each result, then pick WordPress for deeper testing.

Lab question angle: "What web server version is on port 80?" or "Which CMS?" WhatWeb gives both. Then verify with `curl -I`.

Why it works and how defenders stop it: Servers advertise versions in headers and HTML. Defenders hide banners (for example `ServerTokens Prod` in Apache) and remove generator tags.

Memory aid: WhatWeb = "what web stack is this?" Level 1 is quiet, 3 is loud.

3.10 WHOIS

In one line: `whois` shows who registered a domain, when, and which name servers it uses.

Theory in simple words: Every domain has a registration record, like the title deed of a house. WHOIS is the public lookup for that record. Passive: you ask the registry, not the target.

How it works:

1. `whois` contacts the registry or registrar WHOIS server.
2. The server returns the public record.
3. You read the dates, registrar, contacts and name servers.

Hands-on:

`whois` is a Kali default. Purpose: get registration data for `hackersploit.org`.

```
whois hackersploit.org
```

Part	Meaning
whois	The lookup client
hackersploit.org	The registered domain

Output from your notes (shortened and wrapped):

```
Domain Name: HACKERSPLOIT.ORG
Registrar URL: http://www.namecheap.com
Updated Date: 2022-02-05T16:01:46Z
Creation Date: 2018-04-05T11:27:07Z
Registry Expiry Date: 2023-04-05T11:27:07Z
Registrar: NameCheap, Inc.
Registrar IANA ID: 1068
Registrar Abuse Contact Email: abuse@namecheap.com
Registrar Abuse Contact Phone: +1.6613102107
Domain Status: clientTransferProhibited
  https://icann.org/epp#clientTransferProhibited
Registrant Organization: Privacy service provided by Withheld for
  Privacy ehf
Registrant State/Province: Capital Region
Registrant Country: IS
Name Server: JIM.NS.CLOUDFLARE.COM
Name Server: DEE.NS.CLOUDFLARE.COM
DNSSEC: unsigned
```

How to read it:

- **Registrar:** NameCheap, Inc. is the company that sold the domain.
- **Creation / Updated / Expiry:** domain age and when it expires. Expired domains can be taken over.
- **Domain Status clientTransferProhibited:** the registrar lock is on, so the domain cannot be transferred away easily.
- **Registrant Organization:** a privacy service hides the real owner (Capital Region, country IS = Iceland).
- **Name Server:** Cloudflare servers. The site uses Cloudflare DNS, so the real server IP may be hidden.
- **DNSSEC unsigned:** DNS answers are not cryptographically signed.

Useful variations:

```
whois 8.8.8.8
whois hackersploit.org | grep -i "name server"
```

Part	Meaning
whois 8.8.8.8	Look up who owns an IP address block
grep -i	Filter lines, ignoring case

Common mistakes and fixes:

- Querying a subdomain (`www.site.com`). Use the registered domain.
- Expecting names and phones. Privacy services and GDPR (an EU privacy law) hide them.

Worked example: Goal: find the DNS provider for `example.com`. Run `whois example.com | grep -i "name server"`. You see two `NS` names. Write down both, as you will test them for zone transfer in 3.21.

Lab question angle: "Who is the registrar?", "When was the domain created?", "Which are the name servers?" Read the field. Match the date format exactly.

Why it works and how defenders stop it: WHOIS is a public registry requirement. Defenders turn on privacy protection and use role-based contact emails.

Memory aid: whois = who owns, when made, which name servers.

3.11 Netcraft Site Report (including SPF, DMARC and Web Trackers)

In one line: Netcraft is a website that gives a free report on a site's background, hosting, technologies and email security.

Theory in simple words: It is a public dossier on a website, compiled from Netcraft's own scans. You never touch the target, so this is passive recon.

How it works:

1. Open the Netcraft site report page in a browser.
2. Enter the site, such as `https://aniketpandey.website`.
3. Read the sections: Background, Network, hosting, technologies, email security, trackers.

Hands-on: No terminal command. Your notes show two reports.

Report for `https://aniketpandey.website`:

```
Site title:       Aniket Pandey | Full-Stack Developer & Cybersecurity
                  Analyst
Date first seen:  March 2026
Site rank:        Not Present
Primary language: English
Description:      Full-stack web developer and cybersecurity analyst
                  (MERN stack, penetration testing, threat intelligence)
Site:             https://aniketpandey.website
Domain:           aniketpandey.website
```

- **Background** shows the title, first-seen date, rank, language and description. A recent first-seen date can mean a new site.
- **Network** shows the site and domain, and normally hosting details (IP, hosting company, name servers; verify in your report).

Report for `https://hackersploit.org` shows three email and privacy items:

SPF (Sender Policy Framework). An SPF record says which servers may send mail for the domain. Each rule is a qualifier plus a mechanism and argument.

Qualifier	Mechanism	Argument
+ (Pass)	a	my.hackersploit.org
~ (SoftFail)	all	(none)

Reading it: mail from the address of `my.hackersploit.org` passes (+ a). Everything else is a SoftFail (~ all): suspicious but still accepted, usually marked as spam.

> **ADDED CONTEXT:** SPF qualifiers: `+` pass, `-` fail (reject), `~` softfail, `?` neutral. `-all` is stricter than `~all`. Common mechanisms: `a`, `mx`, `ip4`, `include`, `all`.

DMARC (Domain-based Message Authentication, Reporting and Conformance). DMARC tells receiving mail servers what to do when SPF or DKIM checks fail, and where to send reports. The report says: "This host does not have a DMARC record."

Finding: no DMARC record means attackers can more easily send spoofed email that looks like it comes from the domain. This is a weakness to write in a report (phishing risk).

Web Trackers. Third-party resources (social widgets, JavaScript, images) loaded on a page that monitor users for advertising or analytics. The report: "5 known trackers were identified."

Common mistakes and fixes:

- Treating the report as live data. It can lag behind real changes.
- Ignoring email security. SPF and DMARC findings are real report items.

> **ADDED CONTEXT:** You can read SPF and DMARC yourself with `dig TXT` queries. See 3.20.

Worked example: Goal: judge phishing risk for a client domain. The report shows `~all` in SPF and no DMARC. Finding: attackers can spoof the domain. Write down: SPF softfail, DMARC missing.

Lab question angle: "Does the domain have a DMARC record?" or "What qualifier ends the SPF record?" Open the report and read the exact entry.

Why it works and how defenders stop it: Without DMARC, mail servers have no policy to enforce. Defenders publish SPF with `-all`, set up DKIM, and add DMARC with `p=quarantine` or `p=reject`.

Memory aid: SPF = who may send. DMARC = what to do if they should not have.

3.12 DNSDumpster

In one line: DNSDumpster is a free website that lists a domain's DNS records, subdomains, IPs and hosting.

Theory in simple words: You type a domain and it shows a map of the DNS records it knows. It uses stored datasets, so it is passive: the target does not see you.

How it works:

1. Open `https://dnsdumpster.com`.
2. Enter the domain (`aniketpandey.website`) and click "Start Test!".
3. Read the result tables.

Hands-on: No terminal command. Details from your notes:

- Free users are limited to 50 results for a single domain.
- Main table columns: System, Locations, Hosting / Networks, Services / Banners.
- For `aniketpandey.website`: Hosting is CLOUDFLARENET and IONOS-AS - IONOS. Services: cloudflare.
- The **A Records** table (subdomains from dataset) has columns Host, IP, ASN, ASN Name, Open Services (from DB), RevIP.

Terms:

- **ASN:** Autonomous System Number. An ID for a network run by one organization.
- **RevIP:** Reverse IP. Other names pointing to the same IP.
- **Open Services (from DB):** ports seen earlier in a database, not from your own scan.

How to read it: each A record row is a subdomain with its IP and network owner. Subdomains such as `mail`, `dev`, `admin` point to targets to check.

Common mistakes and fixes:

- Treating the result as complete. The 50-result free limit hides some entries.
- Believing "Open Services" is current. Confirm with Nmap (3.26).

Worked example: Goal: list subdomains of `example.com` with no packets. DNSDumpster shows `dev.example.com` at `203.0.113.40`. Decision: check scope, then scan that IP later. Write down: dev subdomain, IP, hosting.

Lab question angle: Questions mostly use terminal tools in the lab because the lab has no internet. In an online-research question, expect "How many subdomains" or "Which provider hosts the site".

Why it works and how defenders stop it: DNS records are public. Defenders avoid naming internal or test systems in public DNS.

Memory aid: DNSDumpster = the DNS map website.

3.13 HTTrack Website Copier

In one line: HTTrack downloads a whole website to your disk so you can browse and search it offline.

Theory in simple words: It is a photocopier for websites. It follows links from the start page, saving HTML, images and other files, and keeps the link structure. You then search the local copy for secrets (comments, config files, flags).

From your notes: HTTrack Website Copier is free (GPL) software, version 3.49-2 (05/20/2017) on the page shown. WinHTTrack is for Windows. WebHTTrack is the Linux, Unix and BSD release (a web interface). It can also update an existing mirror and resume interrupted downloads.

How it works:

1. You give HTTrack a start URL and an output folder.
2. It requests the page, finds links, and downloads each file.
3. It repeats for every internal link.
4. You read the copy with a browser or search it with `grep`.

Hands-on:

On Kali install if missing: `sudo apt install httrack`. HTTrack sends many requests, so it is active recon. Use it only on in-scope targets.

Purpose (CTF Flag 5): mirror the site and search it. Hint in the lab: "Certain files may reveal something interesting when mirrored."

```
httrack http://target.ine.local -O /tmp/mirror
grep -r "flag\|FL@G\|FLAG" /tmp/mirror/
```

Part	Meaning
httrack	The website copier
http://target.ine.local	Start URL to copy
-O /tmp/mirror	Output path: where the copy and logs are saved
grep -r	Search all files in the folder recursively
"flag" "FL@G" "FLAG" parts	Three words joined by backslash-pipe, meaning OR
/tmp/mirror/	The folder to search

Illustrative output (wrapped):

```
Mirror launched on Fri, 13 Nov 2026 10:00:00 by HTTrack Website Copier
mirroring http://target.ine.local with the wizard help..
...
Done.
Thanks for using HTTrack!
```

```
/tmp/mirror/target.ine.local/old/notes.txt:FL@G5{sample-flag-value}
```

How to read the grep line: `path:matching line`. The path before the colon is the file that holds the flag. The notes say the flag is in a mirrored file that normal browsing and dirb may miss, because it is reachable only through internal links.

Useful variations:

```
ls -R /tmp/mirror | head
grep -rniE "flag|FL@G|password" /tmp/mirror/
```

Part	Meaning
ls -R	List folders recursively
head	Show only the first 10 lines
-n	Show line numbers
-i	Ignore upper and lower case
-E	Extended regex: plain pipe means OR, no backslash

> **ADDED CONTEXT:** HTTrack follows `robots.txt` by default, so paths disallowed there may not be copied. The option `-s0` tells it to never follow robots rules (verify with `httrack --help`). Only use it when authorized.

Common mistakes and fixes:

- Mirroring an enormous site. Always use `-O` to a known folder and stop with Ctrl+C if it grows.
- Searching only for "flag". Use several words, including `password`, `key`, `secret`.
- Forgetting `-r` in `grep`. Without it, grep cannot search folders.

Worked example: Goal: find hidden words in `http://10.10.10.5`. Run `httrack http://10.10.10.5 -O /tmp/mirror`. Then `grep -rniE "password|flag" /tmp/mirror/`. A comment in `login.html` reads `TODO remove test password`. Write down: file, line, text.

Lab question angle: "Find the flag in files on the site." Mirror, then grep for the flag format shown in the question.

Why it works and how defenders stop it: Anything linked on a public site can be copied. Defenders keep secrets and comments out of public files and monitor for mass crawling.

Memory aid: HTTrack = take it home, then grep it.

3.14 dnsrecon

In one line: `dnsrecon` is a Python tool that pulls many DNS records for a domain in one run.

Theory in simple words: Instead of asking the phone book one question at a time (as `host` does), `dnsrecon` asks all the common questions and prints the answers together: start of authority, name servers, mail servers, addresses, text records, service records. It queries the target's name servers, so treat it as active recon.

Record names used in the output (full table in 3.19): SOA = start of authority, NS = name server, MX = mail server, A = IPv4 address, TXT = text, SRV = service location.

How it works:

1. You give a domain with `-d`.
2. dnsrecon finds the SOA and NS records.
3. It asks each name server for its software version, then for MX, A, TXT and SRV records.
4. It prints each finding with `[*]` (info), `[+]` (result) or `[-]` (problem).

Hands-on:

`dnsrecon` is on Kali by default. Purpose: enumerate the DNS records of the practice domain `zonetransfer.me`. This domain is a public training target run by DigiNinja for DNS exercises.

```
dnsrecon -d zonetransfer.me
```

Part	Meaning
dnsrecon	The tool
-d	Domain flag: the next word is the target domain
zonetransfer.me	The domain to enumerate

Output from your notes (long lines wrapped, some lines omitted):

```
[*] Performing General Enumeration of Domain: zonetransfer.me
[-] DNSSEC is not configured for zonetransfer.me
[*]  SOA nsztm1.digi.ninja 81.4.108.41
[*]  NS nsztm1.digi.ninja 81.4.108.41
[*]  Bind Version for 81.4.108.41 b"It's a secret"
[*]  NS nsztm2.digi.ninja 34.225.33.2
[*]  Bind Version for 34.225.33.2 b'One more than you'
[*]  MX ASPMX5.GOOGLEMAIL.COM 142.250.157.26
[*]  MX ALT1.ASPMX.L.GOOGLE.COM 142.251.9.26
[*]  MX ASPMX.L.GOOGLE.COM 108.177.119.26
[*]  A zonetransfer.me 5.196.105.14
[*]  TXT zonetransfer.me google-site-verification=tyP28J7JAUHA9fw2sHXMg
     cCC0I6XBmmoVi04VlMewxA
[*] Enumerating SRV Records
[+] {'type': 'SRV', 'name': '_sip._tcp.zonetransfer.me',
     'target': 'www.zonetransfer.me', 'address': '5.196.105.14',
     'port': '5060'}
```

The notes also show more MX lines, including IPv6 addresses (for example `2607:f8b0:400e:c00::1b`).

How to read it:

- `DNSSEC is not configured`: DNS answers are not signed, so spoofing is easier.
- `SOA` and `NS` lines give the name servers `nsztm1.digi.ninja` (81.4.108.41) and `nsztm2.digi.ninja` (34.225.33.2). These are the servers to test for zone transfer (3.21).
- `Bind Version`: the DNS software version string. Here the owners set joke values on purpose, so no real version leaks. On a real target a true version helps you find exploits.
- `MX` lines: mail is handled by Google (ASPMX, GOOGLEMAIL). Several servers give redundancy.
- `A zonetransfer.me 5.196.105.14`: the web server IP.
- `TXT`: a Google site verification token. It proves the domain owner uses Google services.
- `SRV _sip._tcp ... port 5060`: a SIP (voice over IP) service on port 5060 at `www.zonetransfer.me`. SRV records reveal services you would not guess.

Useful variations:

```
dnsrecon -d zonetransfer.me -t axfr
dnsrecon -d example.com -t std
dnsrecon -d example.com -t brt -D /path/to/wordlist.txt
dnsrecon -d example.com -n 10.10.10.2
```

Part	Meaning
-t axfr	Only try a zone transfer against each name server
-t std	Standard record enumeration (the default)
-t brt	Brute-force subdomains from a wordlist
-D file	Wordlist (dictionary) for brute force
-n ip	Use this name server for queries

> **ADDED CONTEXT:** The `-t` type names (std, axfr, brt, srv and others) and the option `-D` are from the dnsrecon help; confirm the full list with `dnsrecon -h` (verify). Wordlist locations differ per Kali version (verify).

Common mistakes and fixes:

- Writing `dnsrecon zonetransfer.me` without `-d`. The domain needs the `-d` flag.
- Reading "Bind Version" as always true. Admins can fake it.
- Running brute force against an out-of-scope domain.

Worked example: Goal: find name servers and mail servers of `example.com`. Run `dnsrecon -d example.com`. You get NS `ns1.example.com 10.10.10.2`. Next, test it with `dnsrecon -d example.com -t axfr`. Write down: NS IP, MX hosts, SRV services.

Lab question angle: "What is the IP of the name server?", "What port does the SRV record list?", "Which mail provider is used?" The values are in the printed lines.

Why it works and how defenders stop it: DNS answers are public for any client that asks. Defenders limit zone transfers, hide the BIND version, and avoid publishing sensitive SRV records.

Memory aid: dnsrecon -d = "DNS recon, domain".

3.15 wafw00f (WAF Detection)

In one line: `wafw00f` detects whether a website sits behind a Web Application Firewall (WAF) and which one.

Theory in simple words: A WAF is a security guard in front of a website. It blocks requests that look like attacks (SQL injection, file inclusion). Knowing a WAF is present tells you that later web attacks may be blocked or need changes. `wafw00f` sends a normal request and then attack-like strings and compares the replies.

How it works:

1. Send a normal request and record the reply (for example 200 OK).
2. Send requests with attack-like strings (SQL injection, file inclusion).
3. Compare headers, cookies and response codes to known WAF signatures.
4. If nothing matches but the replies differ, report a "generic detection".

Hands-on:

`wafw00f` is on Kali (install with `sudo apt install wafw00f` if missing). It is active: it sends attack-like requests.

Purpose: find the WAF of a specific site.

```
wafw00f airtel.in
```

Part	Meaning
wafw00f	The tool
airtel.in	The target site (https is assumed)

Output from your notes:

```
[*] Checking https://airtel.in
[+] The site https://airtel.in is behind Kona SiteDefender (Akamai) WAF.
[~] Number of requests: 2
```

How to read it: `[+]` is a positive finding. The WAF product is named: Kona SiteDefender, which is part of Akamai. `Number of requests` shows how few probes were needed.

Second command from your notes:

```
wafw00f google.com
```

```
[*] Checking https://google.com
[+] Generic Detection results:
[*] The site https://google.com seems to be behind a WAF or some sort
    of security solution
[~] Reason: The server returns a different response code when an
    attack string is used.
    Normal response code is "200", while the response code to a file
    inclusion attack is "404"
[~] Number of requests: 6
```

How to read it: no named product matched, so this is a generic detection. The server answers 200 to a normal request but 404 to a file inclusion test, which suggests something filters attacks. The banner shows `WAFW00F : v2.4.2`, with the tagline "The Web Application Firewall Fingerprinting Toolkit" and a joke list of status codes (404, 405, 403, 502, 500).

Useful variations:

```
wafw00f -a http://10.10.10.5
wafw00f -l
wafw00f -v http://10.10.10.5
```

Part	Meaning
-a	Test all WAF signatures, do not stop at the first match
-l	List all WAFs it can detect
-v	Verbose output

Common mistakes and fixes:

- Concluding "no WAF" when none is found. A tool can miss a custom WAF.
- Scanning a third-party site that is not in scope.

Worked example: Goal: decide whether SQL injection tests will work on `10.10.10.5`. Run `wafw00f http://10.10.10.5`. Result: "No WAF detected". Decision: proceed with normal web tests, keep payloads simple. Write down: no WAF seen.

Lab question angle: "Which WAF protects the site?" or "Is the target behind a WAF?" Answer with the product name printed after `behind`.

Why it works and how defenders stop it: WAFs reveal themselves through headers, cookies and block pages. Defenders can customize block pages, but a determined tester can still tell a filter is present.

Memory aid: wafw00f barks when a WAF is there.

3.16 Sublist3r (Subdomain Enumeration)

In one line: Sublist3r finds subdomains of a domain by asking search engines and public data sources.

Theory in simple words: A subdomain is a prefix on a domain, such as `mail.example.com` or `dev.example.com`. Forgotten subdomains are often weakly protected. Sublist3r collects them from public sources, so your notes label it a passive recon tool, not active (the caption on the notes page).

From your notes: Sublist3r is a Python tool that enumerates subdomains of websites using OSINT (Open Source Intelligence: public data). It uses search engines (Google, Yahoo, Bing, Baidu, Ask) and also Netcraft, Virustotal, ThreatCrowd, DNSdumpster and ReverseDNS. It integrates `subbrute` (by TheRook), which brute-forces names from an improved wordlist.

How it works:

1. You pass a domain.

2. Sublist3r queries each chosen source for names under that domain.
3. It merges and de-duplicates the names.
4. It prints the list (and can save it).

Hands-on:

Sublist3r is available on Kali (`sudo apt install sublist3r` if missing). Purpose: find subdomains from two search engines.

```
sublist3r -d hackersploit.org -e google,yahoo
```

Part	Meaning
sublist3r	The tool
-d hackersploit.org	The domain to enumerate
-e google,yahoo	Engines to use (comma list, no spaces)

Illustrative output (the notes show only the command; wrapped):

```
[-] Enumerating subdomains now for hackersploit.org
[-] Searching now in Google..
[-] Searching now in Yahoo..
[-] Total Unique Subdomains Found: 3
www.hackersploit.org
my.hackersploit.org
academy.hackersploit.org
```

How to read it: the progress lines show each engine used; the final list is your subdomain set. Check each name against the scope (3.2).

Useful variations:

```
sublist3r -d example.com
sublist3r -d example.com -b -t 50
sublist3r -d example.com -o subs.txt
```

Part	Meaning
(no -e)	Use all default sources
-b	Turn on subbrute brute-force (sends DNS queries, so active)
-t 50	Threads for brute force (speed)
-o subs.txt	Save results to a file

> **ADDED CONTEXT:** Other options include `-p` to scan found subdomains for open ports and `-v` for verbose output. Check `sublist3r -h` (verify).

Common mistakes and fixes:

- Search engines may block automated queries (CAPTCHA), so you see fewer results. Try other engines or tools.
- Assuming the list is complete. Combine with DNSDumpster (3.12), dnsrecon (3.14) and theHarvester (3.18).
- Adding spaces in the engine list (`google, yahoo`). Use no spaces.

Worked example: Goal: find web hosts under `example.com`. Run `sublist3r -d example.com -o subs.txt`. You get `www`, `mail`, `dev`. Decision: `dev` may be less hardened. Write down: all names, then resolve each with `host` (3.6).

Lab question angle: "How many subdomains were found?" or "Which subdomain looks like an admin panel?" Use the saved file and count lines with `wc -l subs.txt`.

Why it works and how defenders stop it: Search engines and certificate logs index every public name. Defenders keep an inventory of subdomains and remove old ones.

Memory aid: Sublist3r = subdomain list from the search engines.

3.17 Google Dorking and the Google Hacking Database

In one line: Google dorking is using advanced search operators to find specific pages and files on a target.

Theory in simple words: A normal search finds words. A dork adds filters, like "only this site", "only titles containing this word". It is passive: you query Google, not the target.

Operators from your notes:

Operator	Meaning
site:	Limit results to a site or domain
inurl:	Word must be in the URL
intitle:	Word must be in the page title
*	Wildcard: matches any word or subdomain

> **ADDED CONTEXT:** Other common operators: `filetype:` (file extension), `intext:` (words in the page body), quotes `"..."` (exact phrase) and `-` (exclude). Verify the current list on Google help.

Examples from your notes:

1. Forum pages on a site.

```
site:ine.com inurl:forum
```

Result: 4 results in 0.25 seconds. They are forum and community pages on ine.com (a 2008 forum post, the INE community forum). The pattern: `site:` limits to the domain, `inurl:forum` requires "forum" in the URL.

1. Admin pages on any subdomain.

```
site:*.ine.com intitle:admin
```

Result: about 4 results, such as `my.ine.com` (a Microsoft 365 Teams Admin course) and `community.ine.com` (a post titled "From XSS to Domain Admin"). The `*` makes it match all subdomains. This is also a way to discover subdomains.

1. Open directory listings.

```
intitle:index of
```

Result: about 44,300,000 results such as "Index of /movie" and "Index of /data" with file tables (Parent Directory, file names, dates, sizes). This is the title web servers show when directory listing is enabled. Note: without quotes, Google treats `of` as an ordinary word. A tighter form is `intitle:"index of"`. Add `site:` to limit to your target. Without `site:` you see random sites, which are out of scope.

Google Hacking Database (GHDB): At `https://www.exploit-db.com/google-hacking-database` (the Exploit Database). Your notes show it holds 7,341 entries (at that time) in a table with Date, Dork and Category columns. Categories seen: Web Server Detection, Footholds, Files Containing Juicy Info, Pages Containing Login Portals.

Sample dorks from the notes:

```
inurl:adminpanel site:gov.*
site:gov.* intitle:"index of" *.csv
inurl:/intranet/login.php
intitle:index of settings.py
```

Reading them: each combines operators to find a type of exposed item (admin panels, CSV files in listings, login portals, a Django settings file). Replace `gov.*` with your in-scope domain before use.

How it works:

1. Pick your in-scope domain.
2. Build a dork: `site:yourdomain` plus a filter.
3. Read results. Do not open sensitive third-party data.
4. Save links and findings.

Common mistakes and fixes:

- Running dorks without `site:`. You will find unrelated, out-of-scope data.

- Reading "no results" as "nothing exposed". The search engine may not have indexed it.
- Downloading private data you find. Report it; do not collect more than needed.

Worked example: Goal: find leftover files on `example.com`. Search `site:example.com filetype:bak`, then `site:example.com intitle:"index of"`. You find `https://example.com/old/` with a listing. Write down: URL, file names.

Lab question angle: Knowledge style: "Which operator restricts results to one domain?" -> `site:`. "Which dork finds directory listings?" -> `intitle:"index of"`. The exam lab itself usually has no internet.

Why it works and how defenders stop it: Search engines index everything that is publicly reachable. Defenders use `robots.txt` carefully, disable directory listing, and require login for sensitive files.

Memory aid: SITE narrows, INURL looks in the URL, INTITLE looks in the title.

AI angle (course content, verify if examined): An LLM can suggest dork ideas from a scope description. Example prompt: "List 10 Google dork patterns to find exposed login pages and backup files on example.com, in-scope only, for an authorized test." Always check each dork yourself, and never paste confidential client data into a public AI tool.

3.18 theHarvester

In one line: theHarvester collects emails, names, subdomains and IPs for a domain from many public sources.

Theory in simple words: It is a collector that asks search engines and OSINT services (for example certificate logs) the same question and combines the answers. Email addresses help with phishing tests and username guesses. Hostnames add to your subdomain list.

How it works:

1. You choose a target (`-d`) and sources (`-b`).
2. theHarvester queries each source.
3. It prints sections: emails, hosts, IPs, interesting URLs.
4. Sources that need an API key (a personal access code) print a warning if the key is missing.

Hands-on:

theHarvester is on Kali (`theHarvester`). Banner in your notes: theHarvester 4.7.1, coded by Christian Martorella, Edge-Security Research.

Command 1 from the notes:

```
theHarvester -d INE -b duckduckgo
```

Part	Meaning
theHarvester	The tool (capital H)
-d INE	Target to search (domain or company name)
-b duckduckgo	Data source: the DuckDuckGo search engine

The notes show the banner and `[*] Target: INE`. Note: `INE` is a company name, not a domain. A domain such as `ine.com` gives better results.

Command 2 from the notes (the continuation lines have no indent on purpose, so the shell joins the source list into one word):

```
theHarvester -d zonetransfer.me -b \
duckduckgo,baidu,bing,yahoo,urlscan,crt.sh,intelx,otx,\
rapiddns,whoisxml,tomba
```

Sources in that list:

Source	What it is
duckduckgo, baidu, bing, yahoo	Search engines
urlscan	Public website scan database
crt.sh	Certificate transparency logs: lists names on TLS certificates
intelx	Intelligence X search (needs API key)
otx	AlienVault OTX threat data
rapiddns	DNS records database
whoisxml	WHOIS data service (needs API key)
tomba	Email finder (needs key and secret)

Output from your notes:

```
[*] Target: zonetransfer.me

[!] Missing API key for Intelx.
[!] Missing API key for Tomba Key and/or Secret.
[!] Missing API key for whoisxml.

[*] Searching Bing.
```

How to read it: `[!]` lines are warnings. The three sources with missing keys are skipped, but the others still run. `[*] Searching Bing.` is progress. At the end, the tool lists findings (illustrative):

```
[*] IPs found: 1
5.196.105.14

[*] Hosts found: 3
www.zonetransfer.me
staging.zonetransfer.me
```

Useful variations:

```
theHarvester -d example.com -b crt.sh -l 200
theHarvester -d example.com -b bing -f result
theHarvester -h
```

Part	Meaning
-l 200	Limit the number of results
-f result	Save the report to a file name
-h	Show help and the list of valid sources

> **ADDED CONTEXT:** Some sources (for example baidu or yahoo) have been removed or changed in newer versions. Run `theHarvester -h` to see the current list (verify). API keys go in the tool's `api-keys.yaml` file; the path varies by install (verify).

Common mistakes and fixes:

- Using a company name where a domain is needed.
- Expecting results from keyed sources without keys.
- Using only one source. Combine several and merge the results.

Worked example: Goal: build a host list for `example.com`. Run `theHarvester -d example.com -b crt.sh,bing`. Output: 4 hosts and 2 emails (`info@example.com`, `admin@example.com`). Write down: hosts to check with `host` and emails for the user list.

Lab question angle: "How many email addresses are found?" or "What host was found by crt.sh?" Read the Emails and Hosts sections.

Why it works and how defenders stop it: Public engines and certificate logs record names. Defenders use wildcard certificates, role-based emails, and keep staff addresses off public pages.

Memory aid: -d = what to look for, -b = where to look.

3.19 DNS and DNS Record Types

In one line: DNS (Domain Name System) is the protocol that turns domain names into IP addresses.

Theory in simple words: Early internet users had to remember IP addresses. DNS fixed this by mapping easy names to addresses. A DNS server (name server) is like a telephone directory: you look up a name and get a number. Big public DNS servers hold records for almost every domain, for example Cloudflare 1.1.1.1 and Google 8.8.8.8.

Port: DNS normally uses UDP port 53 and TCP port 53 (zone transfers use TCP).

How it works (a lookup):

```
Your PC -> Resolver (1.1.1.1) -> Root server
                |                    |
                |              TLD server (.com)
                |                    |
                +------------ Authoritative name server
                |
                v
        Answer 203.0.113.10 (cached for the TTL)
```

1. Your PC asks its resolver (for example 1.1.1.1) for `www.example.com`.
2. The resolver asks the root, then the TLD (top-level domain such as .com) server.
3. It reaches the authoritative name server (the one that owns the zone).
4. The answer returns to you and is cached for the TTL (time to live, in seconds).

DNS record types (from your notes):

Record	Meaning	Example data
A	Hostname to IPv4 address	5.196.105.14
AAAA	Hostname to IPv6 address	2606:4700:3031::6815:2cb4
NS	The domain's name server	nsztm1.digi.ninja
MX	Mail server for the domain	10 mx00.ionos.com
CNAME	Alias to another name	www -> example.com
TXT	Free text (SPF, verification)	google-site-verification=...
HINFO	Host information	"Casio fx-700G" "Windows XP"
SOA	Domain authority (zone info)	See below
SRV	Service location and port	_sip._tcp -> port 5060
PTR	IP address to hostname	10.10.10.5 -> web01

SOA fields, using the notes' record for `zonetransfer.me`:

```
nsztm1.digi.ninja. robin.digi.ninja. 2019100801 172800 900 1209600 3600
```

Field	Value	Meaning
Primary NS	nsztm1.digi.ninja.	Main name server
Admin mailbox	robin.digi.ninja.	robin@digi.ninja (first dot is the at sign)
Serial	2019100801	Version number of the zone
Refresh / Retry	172800 / 900	Seconds for secondary servers to check or retry
Expire / Minimum	1209600 / 3600	Seconds before data is stale; negative-cache TTL

Why the pentester cares:

- NS and SOA tell you which servers to test for zone transfer (3.21).
- MX shows the mail provider. TXT shows SPF and third-party services.
- HINFO can leak the operating system.
- CNAME can reveal third-party services.
- SRV reveals service ports (SIP, LDAP, Kerberos).
- PTR (reverse lookup) turns discovered IPs back into names.

Hands-on: The commands that query each record type are in 3.20 (dig) and 3.6 (host).

Worked example: The A record gives a web IP, MX gives the mail host, NS gives where to try AXFR. Write all three in your notes.

Lab question angle: Matching questions: "Which record maps a name to IPv6?" -> AAAA. "Which holds the mail server?" -> MX. "Which reverses an IP to a name?" -> PTR.

Why it works and how defenders stop it: DNS must answer public queries. Defenders split internal and external DNS and avoid publishing internal names.

Memory aid: A-AAAA address; NS name server; MX mail; SOA start of authority; PTR pointer back; SRV service; TXT text; CNAME canonical (alias) name.

3.20 DNS Interrogation and dig

In one line: DNS interrogation means asking a DNS server for the records of a domain; `dig` is the main tool.

Theory in simple words: You question the name server and it replies with what it knows: IP addresses, subdomains and mail server addresses. Tools you have seen (`host`, `dnsrecon`) interrogate DNS. `dig` (Domain Information Groper) is the most flexible: it shows the full reply.

How it works:

1. `dig` builds a DNS query (name, type, class).
2. It sends it to a DNS server (your default, or one you pick with `@`).
3. It prints the header, the question, and the answer, authority and additional sections.

Hands-on:

`dig` is on Kali (package `dnsutils` or `bind9-dnsutils`).

Command from your notes:

```
dig aniketpandey.website all
```

> **TRAP:** `all` is not a `dig` option. `dig` treats it as a second domain name to look up. Your notes show the NXDOMAIN reply for `all.` This is a mistake in the screenshot, not a technique.

Part	Meaning
dig	The lookup tool
aniketpandey.website	First query: the domain (A record by default)
all	A second query for a name called "all" (not an option)

Output from your notes (the second reply, shortened and wrapped):

```
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 38987
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;all.                           IN      A

;; AUTHORITY SECTION:
.            86400   IN   SOA   a.root-servers.net. nstld.verisign-grs.com.
                      2026062400 1800 900 604800 86400

;; Query time: 41 msec
;; SERVER: 1.1.1.1#53(1.1.1.1)
;; WHEN: Wed Jun 24 19:25:01 IST 2026
;; MSG SIZE  rcvd: 107
```

How to read it, line by line:

- `status: NXDOMAIN`: the name does not exist. Other statuses: `NOERROR` (success), `SERVFAIL` (server problem), `REFUSED` (server will not answer).
- `flags: qr rd ra ad`: qr = this is a response, rd = recursion desired, ra = recursion available, ad = authenticated data (DNSSEC checked). `aa` would mean authoritative answer.
- `QUERY: 1, ANSWER: 0`: one question, no answers.
- `QUESTION SECTION`: `all. IN A` shows dig asked for the A record of "all".
- `AUTHORITY SECTION`: the root zone's SOA record, which shows the name does not exist.
- `Query time: 41 msec`, `SERVER: 1.1.1.1#53`: the resolver used and port 53. The earlier reply in the notes shows `Query time: 123 msec` and `MSG SIZE rcvd: 65` for the first query.

Correct ways to query records:

```
dig example.com
dig example.com MX
dig example.com NS
dig example.com TXT +short
dig @8.8.8.8 example.com A
dig -x 10.10.10.5
dig +noall +answer example.com ANY
```

Part	Meaning
example.com	The name to query (A record by default)
MX, NS, TXT	Record type to ask for
+short	Print only the answer data
@8.8.8.8	Ask this DNS server instead of the default
-x 10.10.10.5	Reverse lookup (PTR) of an IP
+noall +answer	Hide all sections except the answer
ANY	Ask for all types; many servers refuse (verify)

Illustrative output of `dig example.com MX +short`:

```
10 mail.example.com.
20 mail2.example.com.
```

Email security checks promised in 3.11:

```
dig TXT example.com +short
dig TXT _dmarc.example.com +short
```

The first shows SPF (a TXT record beginning `v=spf1`). The second shows DMARC (`v=DMARC1`). An empty answer to the second means no DMARC record.

Common mistakes and fixes:

- Adding a bare word like `all` after the domain.
- Forgetting the type for non-A records: use `MX`, `NS`, `TXT`.
- Forgetting `@` before a server name. See 3.21.

Worked example: Goal: map `example.com`. Run `dig example.com NS +short` -> `ns1.example.com.`. Run `dig example.com MX +short` -> `10 mail.example.com.`. Run `dig mail.example.com +short` -> `10.10.10.9`. Write down: NS, MX host, IP.

Lab question angle: "What is the TXT record of the domain?" or "What is the mail server priority?" Use `dig <domain> TXT +short` or `MX +short`.

Why it works and how defenders stop it: Any client may ask a public name server. Defenders limit recursion on authoritative servers and keep private names out of public zones.

Memory aid: dig [@server] name TYPE [+short].

3.21 DNS Zone Transfer (AXFR) with dig

In one line: A zone transfer copies the full DNS zone file from one DNS server to another. If a server allows anyone to ask, you get every record in the zone.

Theory in simple words: A zone is the full list of DNS records a name server holds for a domain. Admins copy zones from a primary server to secondary servers so that DNS keeps working if one fails. The copy request is called AXFR (full zone transfer). If the server is misconfigured and answers any client, an attacker can copy the zone too. Analogy: a bank is supposed to give its full customer list only to its own branch, but the clerk gives it to anyone who asks.

Why it matters (notes): a zone transfer can give a holistic view of the organization's network layout. In some cases internal network addresses are found on the organization's DNS servers.

How it works:

1. Find the name servers (NS records) of the domain.
2. Ask one name server for AXFR over TCP port 53.
3. If it is misconfigured, it sends all records.
4. If it is secure, it refuses and the transfer fails.
5. Repeat for every name server, because each can be set up differently.

Hands-on: Tool first introduced in 3.20 (`dig`). This is active recon: you query the target's own name server.

Command 1 from your notes (a failed attempt):

```
dig axfr brynne.ns.cloudflare.com aniketpandey.website
```

```
; <<>> DiG 9.10.6 <<>> axfr brynne.ns.cloudflare.com aniketpandey.website
;; global options: +cmd
; Transfer failed.
; Transfer failed.
```

> **TRAP:** The server name has no `@` in front. Without `@`, `dig` reads both names as things to look up, which is why `Transfer failed.` prints twice. The correct form is `dig axfr @server domain` (verify behavior on your dig version).

How to read it: `Transfer failed.` means no zone data was returned. For a Cloudflare-hosted domain this is the normal and expected result, because Cloudflare refuses zone transfers.

Command 2 from your notes (success on the practice domain):

```
dig axfr @nsztm1.digi.ninja zonetransfer.me
```

Part	Meaning
dig	The lookup tool
axfr	Query type: full zone transfer
@nsztm1.digi.ninja	Name server to ask (the @ is required)
zonetransfer.me	The domain (zone) to copy

Output from your notes (wrapped; the screenshot shows the top of the zone):

```
; <<>> DiG 9.x.x-Debian <<>> axfr @nsztm1.digi.ninja zonetransfer.me
;; global options: +cmd
zonetransfer.me. 7200 IN SOA nsztm1.digi.ninja. robin.digi.ninja.
    2019100801 172800 900 1209600 3600
zonetransfer.me. 300  IN HINFO "Casio fx-700G" "Windows XP"
zonetransfer.me. 301  IN TXT "google-site-verification=tyP28J7JAUHA9fw
    2sHXMgcCC0I6XBmmoVi04VlMewxA"
zonetransfer.me. 7200 IN MX 0 ASPMX.L.GOOGLE.COM.
zonetransfer.me. 7200 IN MX 10 ALT1.ASPMX.L.GOOGLE.COM.
zonetransfer.me. 7200 IN MX 10 ALT2.ASPMX.L.GOOGLE.COM.
zonetransfer.me. 7200 IN MX 20 ASPMX2.GOOGLEMAIL.COM.
zonetransfer.me. 7200 IN MX 20 ASPMX3.GOOGLEMAIL.COM.
zonetransfer.me. 7200 IN MX 20 ASPMX4.GOOGLEMAIL.COM.
zonetransfer.me. 7200 IN MX 20 ASPMX5.GOOGLEMAIL.COM.
zonetransfer.me. 7200 IN A 5.196.105.14
zonetransfer.me. 7200 IN NS nsztm1.digi.ninja.
zonetransfer.me. 7200 IN NS nsztm2.digi.ninja.
```

How to read it: each line is `name TTL class type data`. `IN` is the Internet class. The transfer returned SOA, HINFO, TXT, MX, A and NS records in one reply. The `HINFO` line says the host is a "Casio fx-700G" running "Windows XP" (a joke value on the training domain; on a real target, HINFO can leak the OS). A real transfer continues with all subdomain records and ends with a record count.

Useful variations:

```
dig NS zonetransfer.me +short
host -l zonetransfer.me nsztm1.digi.ninja
dnsrecon -d zonetransfer.me -t axfr
```

Part	Meaning
dig NS ... +short	Step 1: list name servers to try
host -l domain server	Zone transfer with the host tool
dnsrecon ... -t axfr	Try AXFR against every NS automatically

Common mistakes and fixes:

- Missing `@` before the server (see the trap above).
- Trying only one name server. Test every NS.

- Treating "Transfer failed" as an error in your command. It is often the correct result of a secure server.

Worked example: Goal: test `example.com`. Run `dig NS example.com +short` -> `ns1.example.com.` and `ns2.example.com.`. Run `dig axfr @ns1.example.com example.com` -> refused. Run `dig axfr @ns2.example.com example.com` -> full zone, including `intranet.example.com 10.10.20.5`. Write down: ns2 allows transfers, new internal host and IP. Check scope before using it.

Lab question angle: "What is the IP of the host `internal.<domain>` in the zone?" or "How many A records are in the zone?" Run AXFR against each NS, save the output to a file (`> zone.txt`), and search it with `grep`.

Why it works and how defenders stop it: Zone transfers are a normal admin feature that is left open to everyone by mistake. Defenders allow AXFR only to listed secondary IPs and use TSIG keys (transaction signatures).

Memory aid: AXFR = "give me the whole book". Use `@` to pick the librarian.

3.22 dnsenum

In one line: `dnsenum` is a Perl script that enumerates a domain's DNS: records, zone transfer attempts and subdomain brute force.

Theory in simple words: It combines several steps into one tool: get the main records, find the name servers, try a zone transfer on each, then guess subdomains from a wordlist. Active recon.

How it works:

1. Query A, NS and MX records of the domain.
2. Try a zone transfer on each name server.
3. Optionally scrape Google for names.
4. Brute-force subdomains from a file (default `dns.txt`).
5. Do reverse lookups on the discovered IP ranges (unless `--noreverse`).

Hands-on:

Tool on Kali: `dnsenum`. Your notes show the help screen of dnsenum VERSION 1.2.6 (`dnsenum --help`). Usage line: `dnsenum [Options] <domain>`. If no `-f` is given, it uses `/usr/share/dnsenum/dns.txt` or a `dns.txt` in the same folder.

General options:

Option	Meaning
--dnsserver server	Use this DNS server for A, NS and MX queries
--enum	Shortcut for --threads 5 -s 15 -w
-h, --help	Print help
--noreverse	Skip reverse lookups
--nocolor	Disable colored output
--private	Show and save private IPs in domain_ips.txt
--subfile file	Write all valid subdomains to this file
-t, --timeout value	TCP and UDP timeout in seconds (default 10)
--threads value	Number of threads for queries
-v, --verbose	Show progress and errors

Google scraping options:

Option	Meaning
-p, --pages value	Google pages to process (default 5; -s must be set)
-s, --scrap value	Max subdomains to scrape from Google (default 15)

Brute force options:

Option	Meaning
-f, --file file	Subdomain wordlist (overrides default dns.txt)
-u, --update a, g, r, z	Add valid subdomains to the -f file

`-u` values: `a` = all results, `g` = Google scraping only, `r` = reverse lookup only, `z` = zone transfer only.

> **ADDED CONTEXT:** The `-w` in `--enum` runs whois queries on the network ranges (verify). Your notes show only the help text, so the commands below are standard usage.

Purpose: run the standard enumeration on the training domain.

```
dnsenum zonetransfer.me
dnsenum --enum zonetransfer.me
dnsenum -f /usr/share/dnsenum/dns.txt --dnsserver 10.10.10.2 example.com
```

Part	Meaning
dnsenum zonetransfer.me	Default run with default wordlist
--enum	Shortcut: 5 threads, scrape 15, whois
-f file	Subdomain wordlist for brute force
--dnsserver ip	Ask this server

Illustrative output (the notes show only the tail; wrapped):

```
Host's addresses:
zonetransfer.me.              7200  IN  A      5.196.105.14

Name Servers:
nsztm1.digi.ninja.            7200  IN  A      81.4.108.41
nsztm2.digi.ninja.            7200  IN  A      34.225.33.2

Trying Zone Transfers and getting Bind Versions:
unresolvable name: nsztm1.digi.ninja at /usr/bin/dnsenum line ...
```

From your notes, the end of a successful transfer shows lines such as:

```
...e.   7200   IN   A     5.196.105.14
...e.   300    IN   TXT   "'><script>alert('Boo')</script>"
```

(The left edge of the screenshot is cropped.) The TXT record holds a script tag. This is a test string placed in the zone on purpose. It teaches that DNS data is untrusted input: a web tool that prints DNS records without escaping them could run the script (XSS, cross-site scripting). The last line in your notes, `/usr/share/dnsenum/dns.txt:`, is the default brute-force wordlist path.

Common mistakes and fixes:

- Brute-forcing with a huge list on a slow lab. Use `--threads` carefully.
- Forgetting that `-s` must be set for Google scraping.

Worked example: Goal: collect subdomains of `example.com` and zone data. Run `dnsenum --enum example.com`. Output shows `dev.example.com 10.10.10.40` from the brute force. Write down: host and IP; check scope.

Lab question angle: "Which subdomain resolves to IP X?" Brute force with a wordlist and match.

Why it works and how defenders stop it: It relies on guessable subdomain names and open zone transfers. Defenders use non-obvious names (not a defense by itself), restrict AXFR and rate-limit DNS.

Memory aid: dnsenum = enumerate: records + transfer + brute force.

3.23 fierce

In one line: `fierce` is a DNS reconnaissance tool that locates hostnames and non-contiguous IP space for a domain.

Theory in simple words: Companies own blocks of IP addresses that may not sit next to each other. Fierce uses DNS to find hosts that sit in different parts of the address space, which you can then scan. It is a precursor to scanners like Nmap or Nessus, which need to know what to scan. It does not exploit anything and does not scan the whole internet.

How it works:

1. Find the name servers of the domain and attempt a zone transfer.
2. If that fails, brute-force subdomain names from a wordlist.
3. Look at nearby IP addresses of the hosts found.
4. Report names and IPs.

Hands-on:

Your notes show the older Perl `fierce` (by RSnake, 2006-2007). The screens in your notes:

```
fierce
man fierce
fierce -h
```

```
You have to use the -dns switch with a domain after it.
Type: fierce -h for help
Exiting...
No manual entry for fierce
```

Reading it: running `fierce` alone fails and tells you the syntax is `fierce -dns <domain>`. `man fierce` has no manual page. `fierce -h` shows the help.

Help text from your notes, in plain words:

- Usage: `fierce [-dns example.com] [OPTIONS]`.
- Overview: a semi-lightweight scanner that finds non-contiguous IP space and hostnames. It uses DNS, so it often finds misconfigured networks that leak internal address space.
- `-connect`: try HTTP connections to any non-RFC1918 (public) addresses and print the return headers. The help warns this can take hours to days on a company with many targets.
- RFC1918 = private IP ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.

Purpose: run fierce on a domain (old syntax shown in your notes).

```
fierce -dns zonetransfer.me
```

Part	Meaning
fierce	The tool
-dns	Old-version flag: the domain follows
zonetransfer.me	The target domain

> **ADDED CONTEXT:** Current Kali versions of `fierce` (Python rewrite) use `fierce --domain zonetransfer.me`. Other flags include `--subdomain-file` and `--dns-servers` (verify with `fierce -h`). If one syntax fails, try the other.

Illustrative output (current version, verify):

```
NS: nsztm1.digi.ninja. nsztm2.digi.ninja.
SOA: nsztm1.digi.ninja. (81.4.108.41)
Zone: success
Found: www.zonetransfer.me. (5.196.105.14)
Found: staging.zonetransfer.me. (5.196.105.14)
```

Common mistakes and fixes:

- Using the wrong syntax for your version. Read `fierce -h` first.
- Using `-connect` on a large target. It is very slow.

Worked example: Goal: find IPs near the web server of `example.com`. Run `fierce --domain example.com`. It finds `www` at 10.10.10.5 and `vpn` at 10.10.30.8. Decision: `10.10.30.8` is in a different network block. Check scope and scan it. Write down both IPs.

Lab question angle: "Which subdomain is on a different subnet?" Compare IPs from the output.

Why it works and how defenders stop it: DNS names leak the layout of the network. Defenders use split DNS and avoid descriptive hostnames.

Memory aid: fierce = finds hosts across scattered IP space.

3.24 Host Discovery with Nmap Ping Sweep (nmap -sn)

In one line: `nmap -sn` checks which hosts in a range are alive without scanning ports.

Theory in simple words: Before you scan ports you need to know which IPs have a machine behind them. A ping sweep knocks on every address and notes who answers. Analogy: walking down a street and noting which houses have lights on.

Tool first introduced here: Nmap (Network Mapper). Nmap is the main scanner for the eJPT: it finds hosts, ports, services, OS versions and, with NSE scripts, some vulnerabilities. It lives at `/usr/bin/nmap` on Kali.

How it works:

1. You give a range (CIDR such as `/24` = 256 addresses).
2. On your local network, Nmap sends ARP requests (Layer 2: asks "who has this IP?"). A reply means the host is up and also gives its MAC address.
3. On remote networks, Nmap sends ICMP echo requests (ping) and a few TCP probes (default probes: verify).
4. It prints each host that replied.

Hands-on:

Purpose: list live hosts in a /24 network.

```
sudo nmap -sn 192.168.31.0/24
```

Part	Meaning
sudo	Run as root; needed for raw packets and ARP
nmap	The scanner
-sn	Ping scan: host discovery only, no port scan
192.168.31.0/24	Target range: .0 to .255 (256 addresses)

Output from your notes:

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-24 19:33 IST
Nmap scan report for 192.168.31.1
Host is up (0.012s latency).
MAC Address: 84:28:D6:39:1E:8B (Unknown)

Nmap scan report for 192.168.31.77
Host is up (0.065s latency).
MAC Address: D2:9D:20:43:67:02 (Unknown)

Nmap scan report for 192.168.31.138
Host is up (0.068s latency).
MAC Address: B8:3B:CC:07:99:35 (Xiaomi Communications)

Nmap scan report for 192.168.31.171
Host is up.

Nmap done: 256 IP addresses (4 hosts up) scanned in 4.92 seconds
```

How to read it:

- Each `Nmap scan report` block is one live host.
- `latency` is the reply time. Low latency means a near device.
- `MAC Address ... (vendor)` is the hardware address and maker, shown when ARP was used on your own subnet. `.138` is a Xiaomi device. `.1` is usually the router.
- `192.168.31.171` has no MAC and no latency. This is the machine that ran the scan (a host never ARPs itself). This is an inference.
- The final line: 256 addresses, 4 up.

Useful variations:

```
sudo nmap -sn -n 10.10.10.0/24
sudo nmap -sn 10.10.10.1-20
sudo nmap -sn -iL ranges.txt -oN live.txt
fping -a -g 10.10.10.0/24 2>/dev/null
```

Part	Meaning
-n	No DNS lookups (faster)
10.10.10.1-20	Range from .1 to .20
-iL ranges.txt	Read targets from a file
-oN live.txt	Save normal-format output
fping -a -g range	Alive-only ping sweep with a range
2>/dev/null	Hide errors for unreachable hosts

> **ADDED CONTEXT:** `fping` is another exam tool. `-a` prints only alive hosts and `-g` generates the range. Without root, `nmap -sn` uses TCP connect probes instead of ARP and may miss hosts.

Common mistakes and fixes:

- Running without `sudo` and getting fewer hosts.
- Believing a missing host is down. It may block ping (use `-Pn`, 3.26).
- Sweeping a wide range that is not in scope.

Worked example: Goal: find live hosts in the lab `10.10.10.0/24`. Run `sudo nmap -sn -n 10.10.10.0/24 -oN live.txt`. Result: `.1`, `.5`, `.9` up. Write down the three. Next, port scan each.

Lab question angle: "How many hosts are up on the subnet?" Read the last line of the output. Remember to exclude your own machine and the gateway if the question says so.

Why it works and how defenders stop it: Hosts must answer ARP to work on a LAN. Firewalls can block ICMP and use ARP monitoring to detect sweeps.

Memory aid: -sn = "scan nothing" (no ports), just see who is alive.

3.25 netdiscover

In one line: `netdiscover` finds live hosts on a local network using ARP.

Theory in simple words: ARP (Address Resolution Protocol) is how computers on the same network find each other's MAC addresses. netdiscover sends or listens to ARP traffic to build a list of IPs and MACs. It works only on the same Layer 2 network (same switch/subnet), not across routers.

How it works:

1. You pick an interface and range.
2. The tool broadcasts ARP "who has" requests.
3. Replies fill a live table.
4. In passive mode it only listens for ARP traffic.

Hands-on:

Your notes show that `netdiscover` was missing on that Kali machine. Install it:

```
sudo apt-get install netdiscover -y
```

Part	Meaning
sudo	Run as root
apt-get install	Install a package from the Kali repositories
netdiscover	The package name
-y	Answer yes to prompts automatically

The install output in your notes shows `netdiscover 0.9-1` replacing `0.7-1` and a download of 448 kB. (It also says "6 to remove" and "1724 not upgraded"; these are unrelated package notes.)

Then run it (command from your notes):

```
netdiscover -i eth0 -r 192.168.2.0/24
```

Part	Meaning
netdiscover	The tool
-i eth0	Network interface to use
-r 192.168.2.0/24	Range to scan

The notes show `bash: netdiscover: command not found` again after the install. Likely reasons: the shell did not find it in the PATH because it lives in `/usr/sbin` (run `sudo netdiscover ...` or `/usr/sbin/netdiscover`), or the install and shell hash were out of sync (verify). `netdiscover` needs root anyway.

Illustrative output (the live table):

```
 Currently scanning: Finished!   |   Screen View: Unique Hosts

 3 Captured ARP Req/Rep packets, from 3 hosts.   Total size: 180
 _____________________________________________________________________________
   IP            At MAC Address     Count     Len  MAC Vendor / Hostname
 -----------------------------------------------------------------------------
 192.168.2.1     52:54:00:12:35:00      1      60  Unknown vendor
 192.168.2.10    08:00:27:4a:bb:01      1      60  PCS Systems AG
 192.168.2.15    08:00:27:9e:22:7c      1      60  PCS Systems AG
```

Columns: IP, MAC, count of ARP packets seen, packet length, MAC vendor (maker).

Useful variations:

```
ip a
sudo netdiscover -r 192.168.2.0/24
sudo netdiscover -p -i eth0
```

Part	Meaning
ip a	Shows your interfaces; find the right name
(no -i)	Let netdiscover pick the interface
-p	Passive mode: listen only, send nothing

Common mistakes and fixes:

- Using `eth0` when your lab interface is another name. Check with `ip a`.
- Scanning a remote subnet. ARP does not cross routers.

Worked example: Goal: list devices on the attacker's subnet. `ip a` shows `eth0` and `192.168.2.5/24`. Run `sudo netdiscover -i eth0 -r 192.168.2.0/24`. Three hosts appear. Write down each IP and MAC vendor (a VirtualBox vendor suggests a VM).

Lab question angle: "Which IP has MAC address XX?" or "How many hosts are on the local network?"

Why it works and how defenders stop it: ARP has no authentication. Defenders use dynamic ARP inspection and port security.

Memory aid: netdiscover = ARP radar for the local network.

3.26 Nmap Without Ping: -Pn

In one line: `-Pn` tells Nmap to treat the host as up and skip the host-discovery ping step.

Theory in simple words: By default Nmap pings first. If the host drops those pings (common with Windows Firewall), Nmap decides the host is down and stops. `-Pn` ("no ping") skips the check and goes straight to port scanning.

How it works:

1. Default: Nmap host discovery -> if no reply -> "Host seems down" -> scan stops.
2. With `-Pn`: skip discovery -> scan ports anyway.
3. Nmap scans the default 1000 most common TCP ports.

Hands-on:

Tool first introduced in 3.24. Purpose: show how a firewalled host appears to be down.

```
nmap 10.4.19.218
```

```
Starting Nmap 7.70 ( https://nmap.org ) at 2022-05-04 02:48 IST
Note: Host seems down. If it is really up, but blocking our ping probes,
try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.10 seconds
```

Reading it: Nmap says the host is down but suggests `-Pn`. The host may be up and only blocking pings.

Now with `-Pn`:

```
nmap -Pn 10.4.19.218
```

Part	Meaning
nmap	The scanner
-Pn	Skip host discovery; assume the host is up
10.4.19.218	The target IP

```
Nmap scan report for 10.4.19.218
Host is up (0.0098s latency).
Not shown: 993 filtered ports

PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server
49154/tcp open  unknown
49155/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 4.77 seconds
```

How to read it:

- `Not shown: 993 filtered ports`: 993 ports gave no reply, so a firewall drops them. 993 + 7 open = 1000 ports scanned.
- Port states: `open` (a service answers), `closed` (reachable, nothing listening), `filtered` (no reply: firewall blocks).
- `80 http`: web server. `135 msrpc`: Windows RPC. `139 netbios-ssn` and `445 microsoft-ds`: Windows file sharing (SMB). `3389 ms-wbt-server`: Remote Desktop (RDP). `49154` and `49155`: high dynamic RPC ports, service unknown to Nmap.
- This port set shows a Windows machine.

Common mistakes and fixes:

- Stopping at "host seems down". Retry with `-Pn`.
- Using `-Pn` on a large range. It treats every IP as up and takes much longer.

> **EXAM TIP:** Windows targets in labs often block ping. If a host you expect is "down", try `-Pn` before you give up.

Worked example: Goal: scan `10.10.10.5`. `nmap 10.10.10.5` says down. `nmap -Pn 10.10.10.5` shows 80, 135, 445, 3389 open. Write down: Windows host, ports. Next: version scan.

Lab question angle: "Which port is used for RDP?" -> 3389. "Which ports are open on the target?" Read the PORT table, use `-Pn` if needed.

Why it works and how defenders stop it: Dropping pings only hides the host from basic discovery. Open ports still answer. Defenders use default-deny firewall rules.

Memory aid: -Pn = "Pretend it is up, no ping".

3.27 Fast Scan: -F

In one line: `-F` scans only the 100 most common ports instead of the default 1000.

Theory in simple words: Fewer ports means a faster scan. It is good for a quick first look. It can miss uncommon ports.

How it works: Nmap holds a list of ports ordered by how often they are open. Default scans the top 1000. `-F` scans the top 100.

Hands-on: Purpose: quick scan of the same host.

```
nmap -Pn -F 10.4.19.218
```

Part	Meaning
-Pn	Skip host discovery
-F	Fast mode: top 100 ports
10.4.19.218	Target

```
Nmap scan report for 10.4.19.218
Host is up (0.0093s latency).
Not shown: 93 filtered ports

PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server
49154/tcp open  unknown
49155/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 1.75 seconds
```

How to read it: 93 filtered + 7 open = 100 ports. Same open ports as the default scan, but the time dropped from 4.77 s to 1.75 s.

Useful variations:

```
nmap -Pn -p- 10.10.10.5
nmap -Pn -p 80,443,8080 10.10.10.5
nmap -Pn --top-ports 20 10.10.10.5
```

Part	Meaning
-p-	All 65535 ports
-p 80,443,8080	Only these ports
--top-ports 20	The 20 most common ports

> **ADDED CONTEXT:** On the exam, do a quick scan first and then a full `-p-` scan. A service on an unusual port (for example 8080, 8443 or a high port) is easy to miss with `-F` and is often the way in.

Common mistakes and fixes:

- Trusting `-F` as complete. A closed `-F` result does not mean the port is closed on all 65535.

Worked example: `nmap -Pn -F 10.10.10.5` finds 22 and 80 in two seconds. You then run `nmap -Pn -p- 10.10.10.5` and also find 8080. Write down all three.

Lab question angle: "What non-standard port is open?" Needs a full scan (`-p-`).

Why it works and how defenders stop it: Not a vulnerability. Defenders close unused ports and alert on port scans.

Memory aid: F = Fast, 100 ports.

3.28 Service Version Detection: -sV

In one line: `-sV` asks each open port what software and version is running.

Theory in simple words: Knowing "port 80 is open" is not enough. You need to know "Apache 2.4.49" because versions map to known exploits. `-sV` sends probes to open ports and matches the replies to a signature database.

How it works:

1. Find open ports.
2. For each open port, send probe strings.
3. Match the reply to signatures (product, version, extra info).
4. Add the result to the VERSION column.

Hands-on: Purpose: add version info to the fast scan.

```
nmap -Pn -F -sV 10.4.19.218
```

Part	Meaning
-Pn	Skip host discovery
-F	Top 100 ports
-sV	Probe open ports for service and version

The notes show only the start of this scan (`Starting Nmap 7.70 ( https://nmap.org ) at 2022-05-04 02:59 IST`). Illustrative result (wrapped; versions are not from your notes):

```
PORT      STATE SERVICE       VERSION
80/tcp    open  http          Microsoft IIS httpd 8.5
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds  Microsoft Windows Server 2008 R2 - 2012
                              microsoft-ds
3389/tcp  open  ms-wbt-server Microsoft Terminal Services
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

How to read it: the VERSION column shows the product and version. `Service Info` gives hints about the OS. A CPE is a standard name for a product.

Useful variations:

```
nmap -sV --version-intensity 9 -p 80 10.10.10.5
nmap -sV -p- -oN full_sv.txt 10.10.10.5
```

Part	Meaning
--version-intensity 9	Try harder probes (0 light to 9 all)
-oN file	Save results as text

> **ADDED CONTEXT:** Take the product and version you found and search for known exploits with `searchsploit "Microsoft IIS 8.5"` or in Metasploit. That is what the next modules do.

Common mistakes and fixes:

- Skipping `-sV`. Without it you only see a guessed service name based on port number.
- Trusting the banner. Check by hand if it matters.

Worked example: `nmap -sV -p 21,80 10.10.10.5` -> `21 ftp vsftpd 2.3.4, 80 http Apache 2.2.8`. Write down both versions and search for them.

Lab question angle: "What version of the web server is on port 80?" The answer is in the VERSION column; copy it exactly.

Why it works and how defenders stop it: Services announce themselves. Defenders hide banners and patch.

Memory aid: sV = Service Version.

3.29 OS Detection, Default Scripts and Verbose: -O -sC -v

In one line: `-O` guesses the operating system, `-sC` runs safe default scripts, and `-v` prints more detail while scanning.

Theory in simple words: After finding ports and versions, you want the OS, plus extra facts such as the web page title or SMB (Server Message Block, Windows file sharing) details. Nmap Scripting Engine (NSE) scripts do that.

How it works:

- `-O`: sends special TCP/UDP/ICMP packets and compares the replies to a database of OS fingerprints. Needs root.
- `-sC`: runs the default group of NSE scripts (safe and useful) against open ports.
- `-v`: verbose. Shows open ports as soon as they are found and prints extra scan detail. `-vv` prints even more.

Hands-on: Tool first introduced in 3.24 (Nmap). Purpose: combine fast scan, version detection, OS detection, default scripts and verbose output in one run (command from your notes).

```
nmap -Pn -F -sV -O -sC 10.4.19.218 -v
```

Part	Meaning
nmap	Run as root (on Kali use sudo) because -O needs it
-Pn	Skip host discovery
-F	Top 100 ports
-sV	Service and version detection
-O	OS detection
-sC	Run default NSE scripts
10.4.19.218	Target
-v	Verbose output

The notes show only the start of this scan (`Starting Nmap 7.70 ( https://nmap.org ) at 2022-05-04 03:05 IST`). The screenshot runs it as root (`root@attackdefense`), so `sudo` was not needed there. On a normal Kali user, put `sudo` in front. Illustrative result for a Windows host (not from your notes):

```
Discovered open port 445/tcp on 10.4.19.218
PORT     STATE SERVICE       VERSION
80/tcp   open  http          Microsoft IIS httpd 8.5
|_http-title: Under Construction
445/tcp  open  microsoft-ds  Windows Server 2012 R2 microsoft-ds
3389/tcp open  ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
|   Target_Name: WIN-LAB01
|_  DNS_Computer_Name: WIN-LAB01
Device type: general purpose
Running: Microsoft Windows 2012|8.1
OS details: Microsoft Windows Server 2012 R2
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

How to read it:

- `Discovered open port ...` lines appear live because of `-v`.
- Lines starting with `|` or `|_` are NSE script results (for example `http-title` shows the page title).
- `Running` and `OS details` are the OS guess. Nmap may list several candidates. Treat it as a guess and confirm with other facts (ports 135, 445, 3389 point to Windows).
- `rdp-ntlm-info` can reveal the computer name.

Useful variations:

```
sudo nmap -A -Pn 10.10.10.5
sudo nmap -sC -sV -p- -oN full.txt 10.10.10.5
nmap --script smb-os-discovery -p 445 10.10.10.5
```

Part	Meaning
-A	Aggressive: -sV, -O, -sC and traceroute together
-p-	All 65535 ports
-oN full.txt	Save normal output
--script name	Run one named NSE script
-p 445	Limit to port 445

Common mistakes and fixes:

- Running `-O` without root. It will fail or give nothing.
- Trusting the OS guess when Nmap says "too many fingerprints". Cross-check with ports and banners.
- Running default scripts on a fragile live system. Use only on in-scope lab targets.

Worked example: Goal: profile `10.10.10.5` fully. Run `sudo nmap -Pn -sV -O -sC -p- -oN full.txt 10.10.10.5`. Read the file. It shows Windows Server, SMB, RDP and IIS. Write down: OS guess, each port and version, script findings (computer name). Next: pick the weakest service to attack.

Lab question angle: "What operating system does the host run?" or "What is the page title of the web server?" Use `-O` and `-sC`, then copy the exact text.

Why it works and how defenders stop it: TCP/IP stacks behave a little differently per OS, and services leak details. Defenders normalize stack behavior with firewalls and hide banners.

Memory aid: O = Operating system. sC = Scripts, Common (default). v = more Voice.

3.30 Reading Page Source: the HTML Generator Meta Tag

In one line: Many CMS platforms write their name and version into an HTML `meta generator` tag, and you can pull it out with `curl` and `grep`.

Theory in simple words: A web page is HTML text. The `<head>` may hold a tag like `<meta name="generator" content="WordPress 6.5.3" />`. It tells anyone who views the source which software built the page. The lab hint is: "What website is running on the target, and what is its version?"

Key lesson from your notes: `curl -I` only shows the HTTP headers (here only the Apache `Server:` header). The generator is in the body, so you must fetch and search the full page.

How it works:

1. `curl` downloads the full HTML body.
2. The pipe `|` sends that text into `grep`.
3. `grep` prints only the lines that match the word.

Hands-on: Tools: `curl` (first introduced in 3.7) and `grep` (a text search tool, standard on Linux).

Purpose (CTF Flag 2): find the CMS and version.

```
curl -s http://target.ine.local | grep -i "generator"
```

Part	Meaning
curl	Fetch the page
-s	Silent: hide the progress meter so only HTML is passed on
http://target.ine.local	Target URL
the pipe character	Send curl's output into the next command
grep	Print lines matching a pattern
-i	Ignore upper and lower case
"generator"	The word to find

Output from your notes:

```
<meta name="generator" content="WordPress 6.5.3 -
  FL@G2{46308b14a4204a99adc2f08917a4b0a1}" />
```

(The real output is one long line. It is wrapped here for print.)

How to read it: `name="generator"` marks the tag. The `content` value names the CMS (WordPress) and version (6.5.3). On the lab target, the flag is also placed inside the content string.

Useful variations:

```
curl -I http://target.ine.local
curl -s http://target.ine.local | grep -i "<meta"
curl -s http://target.ine.local | grep -iE "generator|wp-content|version"
curl -s http://target.ine.local | grep -o "wp-content/[^\"']*" | head
```

Part	Meaning
curl -I	Headers only; shows the Server banner
grep -i "<meta"	All meta tags
-E	Extended regex: words joined by pipes match any
grep -o	Print only the matching text
head	Show the first 10 lines

Common mistakes and fixes:

- Using `curl -I` and stopping at the `Server: Apache` header. Always search the full body.
- Typing the URL without quotes when it has `&` or `?`. Quote it.
- The generator tag may be removed by the owner. Use other clues: `/wp-login.php`, `/wp-content/` paths, `robots.txt`.

Worked example: Goal: identify the CMS. `curl -I http://10.10.10.5` shows `Server: Apache/2.4.52`. `curl -s http://10.10.10.5 | grep -i generator` shows `Joomla! - Open Source Content Management`. Write down: Joomla, Apache 2.4.52.

Lab question angle: "What CMS and version does the site use?" The answer is in this tag. Copy the exact version number.

Why it works and how defenders stop it: The CMS adds the tag by default. Defenders remove it with a security plugin or a code snippet and keep the CMS patched.

Memory aid: Headers are the envelope. The generator tag is in the letter, so read the letter.

3.31 Directory Brute-Forcing with dirb

In one line: `dirb` tries thousands of common folder and file names on a web server and shows which ones exist.

Theory in simple words: Many sites have pages that are not linked anywhere (admin panels, backups, old uploads). A brute-force scanner guesses names from a wordlist and checks the HTTP status code of each guess. Analogy: trying every door handle in a corridor. The lab hint: "Directory browsing might reveal where files are stored."

Directory browsing (listing) is when a web server shows the folder contents as a file list because no index page exists. It is a misconfiguration that lets you see and open all files in that folder.

How it works:

1. dirb loads a wordlist (default `/usr/share/dirb/wordlists/common.txt`, about 4600 words; verify path on your Kali).
2. For each word it requests `URL/word` and `URL/word/`.
3. It prints the hits with status codes. 200 = exists. 301/302 = redirect (often a directory). 403 = exists but forbidden. 404 = not found.
4. It recurses into found directories.

Hands-on:

`dirb` is on Kali (`dirb`). It is active recon and is noisy: thousands of requests. Purpose (CTF Flag 3): find directories and save the result.

```
dirb http://target.ine.local -w > dirb.txt
cat dirb.txt
```

Part	Meaning
dirb	The scanner
http://target.ine.local	Base URL to scan
-w	Do not stop when a WARNING appears
> dirb.txt	Redirect the output into the file dirb.txt
cat dirb.txt	Print the saved file

Illustrative output (not from your notes):

```
-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Fri Nov 13 10:00:00 2026
URL_BASE: http://target.ine.local/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
OPTION: Not Stopping on warning messages

---- Scanning URL: http://target.ine.local/ ----
+ http://target.ine.local/index.php (CODE:301|SIZE:0)
+ http://target.ine.local/xmlrpc.php (CODE:405|SIZE:42)
==> DIRECTORY: http://target.ine.local/wp-admin/
==> DIRECTORY: http://target.ine.local/wp-content/
==> DIRECTORY: http://target.ine.local/files/
+ http://target.ine.local/server-status (CODE:403|SIZE:278)
```

How to read it:

- `+ URL (CODE:xxx|SIZE:n)` is a file hit with its status and size.
- `==> DIRECTORY:` is a folder. Open it in Firefox or with `curl`.
- `wp-admin` and `wp-content` confirm WordPress (as your notes say: dirb confirmed a WordPress installation).
- `files/` is a candidate for directory listing. Open it:

```
curl http://target.ine.local/files/
```

Look for `Index of /files` and file links. Then fetch a file with `curl http://target.ine.local/files/<name>`. The flag is inside a listed file.

Useful variations:

```
dirb http://10.10.10.5 /usr/share/wordlists/dirb/big.txt
dirb http://10.10.10.5 -X .php,.bak,.txt
dirb http://10.10.10.5 -r -o scan.txt
dirb http://10.10.10.5 -w | tee dirb.txt
```

Part	Meaning
second argument (path)	Use a different wordlist
-X .php,.bak,.txt	Also try these file extensions
-r	Not recursive (do not enter found folders)
-o scan.txt	Save output to a file
tee dirb.txt	Show on screen and save to a file

> **ADDED CONTEXT:** The redirect `> dirb.txt` hides all output on screen. Use `| tee dirb.txt` to watch it live. dirb can be slow; wordlist paths differ per Kali version (verify).
> **ADDED CONTEXT:** Gobuster is the faster tool the exam update names for directory brute forcing: `gobuster dir -u http://10.10.10.5 -w /usr/share/wordlists/dirb/common.txt -x php,bak,txt -t 20`. Parts: `dir` = directory mode, `-u` target URL, `-w` wordlist, `-x` extensions, `-t` threads. Output lines look like `/wp-admin (Status: 301) [Size: 315]`. Verify flags with `gobuster dir -h`.

Common mistakes and fixes:

- Forgetting the trailing path. Use the base URL of the correct server.
- Reading only 200 results. A 403 directory may still hold readable files.
- Using the wrong protocol (`http` vs `https`).

Worked example: Goal: find hidden folders on `10.10.10.5`. Run `dirb http://10.10.10.5 -w | tee dirb.txt`. It finds `/backup/` and `/wp-content/`. `curl http://10.10.10.5/backup/` shows `Index of /backup` with `site.zip`. Write down: path, file names. Next, download and inspect the file.

Lab question angle: "Which hidden directory exists on the web server?" or "Find the flag in the directory listing." Brute-force, open each folder, read file names.

Why it works and how defenders stop it: Servers answer for any path, and listings are enabled by mistake. Defenders turn off directory indexing (`Options -Indexes` on Apache), remove old files, and use rate limits.

Memory aid: dirb = directory bruteforce; 200 and 403 mean the path is real.

3.32 Backup Files in the Web Root

In one line: Editors and admins leave copies like `wp-config.php.bak` on the server, and a web server shows these as plain text.

Theory in simple words: `wp-config.php` is the main WordPress settings file. It holds the database name, user, password and security keys. The server runs `.php` files, so you see nothing if you open it. A copy named `wp-config.php.bak` does not end in `.php`, so the server sends it as text and the secrets are exposed. The lab hint: "An overlooked backup file in the webroot can be problematic."

Common backup suffixes (from your notes): `.bak`, `.old`, `.orig`, `.backup`, `~` (editor backup), `.save`.

How it works:

1. Confirm the site is WordPress (dirb found `wp-admin`, 3.31).

2. Request `wp-config.php` plus each suffix.
3. Print only the HTTP status code of each attempt.
4. The suffix that returns 200 (OK) exists. Read that file.

Hands-on: Tools: `curl` (3.7) and the shell `for` loop. Purpose (CTF Flag 4, step 1): find which backup suffix exists.

```
for ext in .bak .old .orig .backup '~' .save; do
  code=$(curl -s -o /dev/null -w "%{http_code}" \
    http://target.ine.local/wp-config.php$ext)
  echo "$code -> wp-config.php$ext"
done
```

Part	Meaning
for ext in LIST; do ... done	Repeat the body once per suffix in the list
ext	Loop variable that holds the current suffix
'~'	The tilde, quoted so the shell does not expand it
code=$(...)	Run the command and store its output in code
curl -s	Silent mode
-o /dev/null	Throw away the page body
-w "%{http_code}"	After the request, print only the HTTP status code
wp-config.php$ext	The URL path with the current suffix added
echo "$code -> ..."	Print status code and file name together

> **TRAP:** Your notes list the tilde as a bare `~`. In bash, an unquoted `~` expands to your home directory (for example `/root`), so that test would request the wrong URL. Quote it as `'~'`.

Illustrative output:

```
200 -> wp-config.php.bak
404 -> wp-config.php.old
404 -> wp-config.php.orig
404 -> wp-config.php.backup
404 -> wp-config.php~
404 -> wp-config.php.save
```

How to read it: only `.bak` returns 200, so that file exists and can be read. 403 means it exists but is blocked. 404 means not found.

Step 2 from your notes: read the file that returned 200.

```
curl http://target.ine.local/wp-config.php.bak
```

Illustrative output (values are fake):

```
<?php
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
define( 'DB_PASSWORD', 'S3cretPass!' );
define( 'DB_HOST', 'localhost' );
// FL@G4{sample-flag-value}
```

On the lab, your notes say the flag is embedded in the backup file content. In a real test, the database password is the real prize: it may be reused for other logins.

Useful variations:

```
curl -s -o /dev/null -w "%{http_code}\n" http://10.10.10.5/db.sql
dirb http://10.10.10.5 -X .bak,.old,.zip,.sql
```

Part	Meaning
\n in the format	Print a newline after the code
-X .bak,.old,.zip,.sql	dirb tries these extensions on every word

> **ADDED CONTEXT:** WPScan (a WordPress scanner) can look for config backups with `--enumerate cb` (verify). Other files worth guessing: `backup.zip`, `db.sql`, `.git/`, `.env`, `config.php.txt`.

Common mistakes and fixes:

- Opening `wp-config.php` itself and seeing a blank page. PHP runs on the server. Only copies show the source.
- Forgetting to quote `'~'` (see the trap).
- Testing only one suffix. The loop tests all.

Worked example: Goal: look for leaked config. dirb shows WordPress. The loop prints `200 -> wp-config.php.bak`. `curl` the file and read `DB_USER` and `DB_PASSWORD`. Write down: file name, user, password, host. Test the password later on SSH or the admin panel (only if in scope).

Lab question angle: "What is the database password in the config backup?" or "Find the flag in the backup file." Loop over suffixes, then read the 200 file.

Why it works and how defenders stop it: The server serves any file that is not mapped to a script. Defenders keep backups outside the web root, block these suffixes, and never edit files live on the server.

Memory aid: If .php hides it, a .bak shows it.

3.33 CTF Walkthrough: Information Gathering CTF 1

In one line: The notes end with a five-flag lab on `http://target.ine.local` that uses only curl, dirb and HTTrack.

Theory in simple words: Each flag has a hint in plain words. The hint names the technique. This block shows how each hint maps to a technique that is explained fully in the earlier blocks. The result recorded in your notes: 5 of 5 flags captured.

Hint-to-technique map:

Flag	Hint (from the notes)	Technique	Block
1	Tells search engines what to avoid	Read robots.txt	3.7
2	What website and which version	Meta generator tag	3.30
3	Directory browsing may show files	dirb, open listings	3.31
4	Overlooked backup file in webroot	Backup suffix loop	3.32
5	Mirrored files may reveal something	HTTrack, then grep	3.13

How it works (the method that finds all five):

1. Read the hint. It names the weakness.
2. Run the one command that tests that weakness.
3. Read the whole output, not only the first line.
4. Copy the flag exactly.
5. Move on. Note which technique worked.

What each flag teaches:

- Flag 1: always read `robots.txt` in full. The flag was inside the file.
- Flag 2: headers (`curl -I`) show only `Apache`. The version and the flag are in the page body. Search the full page.
- Flag 3: brute-force with dirb, look for 200 or 403 directories with open listings, then open them.
- Flag 4: dirb showed WordPress, so a `wp-config.php` backup was a smart guess. Test suffixes by status code.
- Flag 5: HTTrack mirrors files reachable only through internal links, which dirb and normal browsing can miss. Then `grep -r` the mirror.

Worked example (new target, same order): Goal: sweep a new web server quickly. Check `robots.txt`, grep the generator, run dirb, test backups on the config file you expect, mirror and grep. Write each result next to its technique in your notes.

Lab question angle: Flag formats differ. Your notes show `FL@G2{...}` for flag 2. Always use the format given in the question.

Why it works and how defenders stop it: Each flag is one common misconfiguration: a revealing robots file, a version tag, directory listing, a stray backup, a secret in a linked file. A short checklist and a content review prevent all five.

Memory aid: R-G-D-B-H: Robots, Generator, Dirb, Backup, HTTrack.

4. Chained Scenarios

All IPs and names below are fake lab values. Each step gives the command and a one-line reason. Full explanations are in the block named in brackets.

Scenario A: External domain, passive to active

Goal: map `example.com` and find a leaked internal host. Scope: `example.com`, `*.example.com`.

Step 1. Confirm the scope entry before anything else (see 3.2). Write the scope in your notes.

Step 2. Registration data, no packets to the target (see 3.10).

```
whois example.com | grep -i "name server"
```

Step 3. Addresses and mail servers (see 3.6).

```
host example.com
host -t mx example.com
```

Step 4. Email security gaps for the report (see 3.20).

```
dig TXT example.com +short
dig TXT _dmarc.example.com +short
```

Step 5. More subdomains from public sources (see 3.16, 3.18).

```
sublist3r -d example.com -o subs.txt
theHarvester -d example.com -b crt.sh,bing
```

Step 6. Active: try a zone transfer on every name server (see 3.21).

```
dig NS example.com +short
dig axfr @ns2.example.com example.com
```

Step 7. Result: `ns2` leaks `intranet.example.com 10.10.20.5`. Check scope again, then resolve the new names and scan the web host (see 3.26, 3.28).

```
nmap -Pn -sV 10.10.20.5
```

Notes to save: NS list, which NS allowed AXFR, the internal IP, open ports and versions.

Scenario B: Internal lab network to a profiled web server

Goal: find live hosts in `10.10.10.0/24`, then profile one web server. Scope: that /24.

Step 1. Find live hosts (see 3.24).

```
sudo nmap -sn -n 10.10.10.0/24 -oN live.txt
```

Step 2. One host looks down but you expect a Windows machine. Skip ping (see 3.26).

```
nmap -Pn 10.10.10.9
```

Step 3. Quick scan, then a full port scan on the interesting host (see 3.27).

```
nmap -Pn -F 10.10.10.5
nmap -Pn -p- -oN allports.txt 10.10.10.5
```

Step 4. Versions, OS and default scripts on the open ports only (see 3.28, 3.29).

```
sudo nmap -Pn -sV -O -sC -p 22,80,8080 10.10.10.5
```

Step 5. Fingerprint the web stack and check for a WAF (see 3.9, 3.15).

```
whatweb -a 1 http://10.10.10.5
wafw00f http://10.10.10.5
```

Step 6. Quick web checks (see 3.7, 3.30).

```
curl http://10.10.10.5/robots.txt
curl -s http://10.10.10.5 | grep -i "generator"
```

Step 7. Hidden content (see 3.31).

```
dirb http://10.10.10.5 -w | tee dirb.txt
```

Notes to save: live hosts, open ports per host, versions, CMS, hidden paths. These feed the exploitation modules.

Scenario C: Web target flag hunt (CTF style)

Goal: collect five flags from `http://target.ine.local`. Same method as 3.33.

Step 1. Robots file (see 3.7).

```
curl http://target.ine.local/robots.txt
```

Step 2. Generator tag (see 3.30).

```
curl -s http://target.ine.local | grep -i "generator"
```

Step 3. Brute-force directories, then open listings (see 3.31).

```
dirb http://target.ine.local -w > dirb.txt
cat dirb.txt
```

Step 4. WordPress confirmed, so test config backups (see 3.32).

```
for ext in .bak .old .orig .backup '~' .save; do
  code=$(curl -s -o /dev/null -w "%{http_code}" \
    http://target.ine.local/wp-config.php$ext)
  echo "$code -> wp-config.php$ext"
done
```

Step 5. Mirror the site and search it (see 3.13).

```
httrack http://target.ine.local -O /tmp/mirror
grep -r "flag\|FL@G\|FLAG" /tmp/mirror/
```

5. Decision Guide and Exam Strategy

If I see X, I try Y

If I see	I try	Block
A domain in scope	whois, host, dig NS MX TXT	3.10, 3.6, 3.20
Name servers listed	dig axfr @ns domain, for each NS	3.21
A /24 range in scope	sudo nmap -sn -n range	3.24
Host says "seems down"	nmap -Pn host	3.26
Open ports, no names	nmap -sV on those ports	3.28
Ports 135 139 445 3389	Windows target; note SMB and RDP	3.26
Port 80 or 443 open	whatweb, robots.txt, generator, dirb	3.9, 3.7, 3.30, 3.31
WordPress paths seen	Check wp-config.php backups	3.32
Many subdomains found	Resolve each, check scope, scan	3.6, 3.2
Need OS or scripts	sudo nmap -O -sC	3.29
Web attacks may be blocked	wafw00f first	3.15
Site mirror might hold secrets	httrack, then grep -r	3.13
Local subnet, MAC needed	netdiscover or nmap -sn	3.25
TXT record or SPF question	dig TXT domain +short	3.20

Enumeration checklist for this module

- Write the scope and re-check every new name or IP against it.
- Passive: whois, host, dig (A, NS, MX, TXT), Netcraft, DNSDumpster, dorks, theHarvester, Sublist3r.
- DNS: zone transfer against every name server; dnsrecon or dnsenum for records and brute force.

- Hosts: `nmap -sn`, then `-Pn` for silent hosts, netdiscover on the local subnet.
- Ports: quick `-F`, then `-p-` on important hosts.
- Services: `-sV`, then `-O -sC` on interesting hosts.
- Web: WhatWeb, robots.txt and sitemap.xml, generator tag, wafw00f, dirb or Gobuster, backup files, HTTrack mirror and grep.
- Save every command, IP, port, version, user, file name and flag.

Common traps

> **TRAP:** Stopping at "Host seems down". Many lab hosts block ping. Use `-Pn`.
>
> **TRAP:** Missing `@` in `dig axfr @server domain`. Your own notes show the failed form (3.21).
>
> **TRAP:** Scanning only the top 100 ports and missing a service on a high port. Run `-p-` on key hosts.
>
> **TRAP:** Unquoted `~` in the backup loop expands to your home directory (3.32).
>
> **TRAP:** Using `curl -I` only. Versions and flags often live in the page body (3.30).

Other traps:

- Typing `dnsrecon zonetransfer.me` without `-d`.
- Trusting the Nmap OS guess or a banner version without cross-checking.
- Scanning out-of-scope subdomains that tools found.
- Forgetting `sudo` for `-sn` on the local network and for `-O`.
- Sources of theHarvester that need an API key give no results without it.
- Redirecting dirb with `>` and then waiting for output that does not show on screen.

How to take notes in the lab

Keep one text file per target and one master file. Use the same fields every time.

```
HOST     : 10.10.10.5   (scope: yes)
OS GUESS : Windows Server 2012 R2 (nmap -O)
PORTS    : 80 IIS 8.5 | 445 SMB | 3389 RDP
WEB      : WordPress 6.5.3, no WAF, /files/ listing
DNS      : NS ns1 (AXFR denied) ns2 (AXFR ok)
CREDS    : wpuser / S3cretPass!  (from wp-config.php.bak)
FLAGS    : FL@G4{...}  source: wp-config.php.bak
NEXT     : test SMB shares, try wpuser password on admin login
```

Save raw command output too: `nmap ... -oN file.txt`, `dirb ... | tee file.txt`, `dig axfr ... > zone.txt`. With files you can `grep` instead of re-running scans. Use a screenshot or a copied output line as evidence for report-style questions.

Turning findings into answers

Multiple choice. Find the value in the lab first, then match it to the option. Do not pick from memory when you can check. Eliminate options that break a rule (for example, a passive method that sends packets).

Dynamic-value or flag questions. The answer is a string only your lab instance holds (a version, a port, a flag, a password). Copy it from the tool output or file. Check spaces, case and trailing characters. If a flag format is given in the question (for example `FL@G{...}`), match it exactly.

Fill-in-the-blank. Use the exact term the tool prints (for example `ms-wbt-server`, `microsoft-ds`, `NXDOMAIN`). When the question asks for a count, count from your saved file (`wc -l`, `grep -c`) instead of by eye.

> **EXAM TIP:** If a question mentions a host you never scanned, scan it now, save the result, and answer from the scan. Questions can only be answered from what you actually find in the lab.
>
> **EXAM TIP:** Spend the first minutes on breadth: ping sweep, quick scan, robots and generator on every web host. Go deep only on hosts that show promise.

6. Practice

All data is fake and for practice. Run the commands against your own lab only.

6.1 Lab-style tasks

Task 1: Scope check

Scenario: The scope says `app.corp.local` and `10.10.10.0/24`. Your tools found `10.10.10.77`, `10.10.11.4`, `dev.corp.local`, `cdn.vendor.net`.

Do: compare each item to the scope text. `/24` fixes the first three numbers.

Expect: `10.10.10.77` in scope. `10.10.11.4` out. `cdn.vendor.net` out (third party). `dev.corp.local` is not listed, so ask or report it before touching it.

Answer: write in/out beside each item in your notes (see 3.2).

Task 2: Addresses and mail with host

Scenario: Find the IPv4 address and mail server of `lab.example.com`.

```
host lab.example.com
host -t mx lab.example.com
```

Expect:

```
lab.example.com has address 10.10.10.5
lab.example.com mail is handled by 10 mail.lab.example.com.
```

Answer: A line gives the IP. The MX line gives the mail host and priority (see 3.6).

Task 3: robots.txt secrets

Scenario: A web server at `http://10.10.10.5` hides an admin area. Find the disallowed paths.

```
curl -s http://10.10.10.5/robots.txt
curl -s http://10.10.10.5/sitemap.xml
```

Expect:

```
User-agent: *
Disallow: /admin-old/
Disallow: /private/
```

Answer: the two Disallow paths. Visit them with `curl -I http://10.10.10.5/admin-old/` to see the status code (see 3.7).

Task 4: Registrar and name servers

Scenario: Report the registrar, creation date and name servers of `example.org`.

```
whois example.org | grep -iE "registrar:|creation|name server"
```

Expect:

```
Registrar: Example Registrar, Inc.
Creation Date: 2015-03-12T09:00:00Z
Name Server: NS1.EXAMPLE.ORG
Name Server: NS2.EXAMPLE.ORG
```

Answer: copy the values exactly, including the date format (see 3.10).

Task 5: Identify the CMS

Scenario: Which web technology and version run on `http://10.10.10.5`?

```
whatweb -a 1 http://10.10.10.5
curl -s http://10.10.10.5 | grep -i "generator"
```

Expect: a line with `WordPress[6.5.3]` and a meta tag `content="WordPress 6.5.3"`.

Answer: `WordPress 6.5.3`. Two tools agree, so the result is reliable (see 3.9, 3.30).

Task 6: WAF check

Scenario: Decide if a WAF protects `https://shop.example.com`.

```
wafw00f https://shop.example.com
```

Expect: `The site ... is behind Cloudflare (Cloudflare Inc.) WAF.` or `No WAF detected by the generic detection`.

Answer: the product name after `behind`, or "none seen" (see 3.15).

Task 7: SPF and DMARC

Scenario: Does `example.com` allow spoofed mail?

```
dig TXT example.com +short
dig TXT _dmarc.example.com +short
```

Expect:

```
"v=spf1 +a ~all"
```

and an empty answer for the DMARC query.

Answer: SPF ends in `~all` (softfail) and DMARC is missing, so spoofing is easier (see 3.11, 3.20).

Task 8: Subdomain list

Scenario: Build a subdomain list for `example.com` from public sources, then count it.

```
sublist3r -d example.com -o subs.txt
theHarvester -d example.com -b crt.sh -f harvest
wc -l subs.txt
```

Expect: a file with names such as `www`, `mail`, `dev`; `wc -l` prints the count.

Answer: merge, remove duplicates with `sort -u`, and check scope (see 3.16, 3.18).

Task 9: Write a dork

Scenario: Find directory listings and admin titles on `ine.com` only.

```
site:ine.com intitle:"index of"
site:*.ine.com intitle:admin
```

Expect: pages in Google results restricted to the domain.

Answer: `site:` restricts to the domain, `intitle:` searches page titles (see 3.17).

Task 10: SRV record port

Scenario: Which port does the SIP service use in `zonetransfer.me`?

```
dnsrecon -d zonetransfer.me
```

Expect: a line `_sip._tcp.zonetransfer.me ... 'port': '5060'`.

Answer: `5060` (see 3.14).

Task 11: Zone transfer

Scenario: Find which name server of `zonetransfer.me` allows AXFR and read the A record.

```
dig NS zonetransfer.me +short
dig axfr @nsztm1.digi.ninja zonetransfer.me
```

Expect: the full zone, including `zonetransfer.me. 7200 IN A 5.196.105.14`.

Answer: `5.196.105.14`. Always test both NS servers (see 3.21).

Task 12: Fix a broken zone transfer command

Scenario: A teammate typed `dig axfr ns1.example.com example.com` and got `Transfer failed.` twice.

Do: add the missing `@`.

```
dig axfr @ns1.example.com example.com
```

Expect: either the zone, or a single refusal such as `Transfer failed` / `; Transfer failed.` from a secure server.

Answer: the server must follow `@` (see 3.21).

Task 13: Subdomain brute force

Scenario: Find a hidden subdomain of `example.com` with a wordlist.

```
dnsenum -f /usr/share/dnsenum/dns.txt --enum example.com
```

Expect: a "Brute forcing" section with lines such as `dev.example.com. 300 IN A 10.10.10.40`.

Answer: the host that resolves and its IP. Check scope (see 3.22).

Task 14: Count live hosts

Scenario: How many hosts are alive in `192.168.31.0/24`?

```
sudo nmap -sn 192.168.31.0/24
```

Expect: last line `Nmap done: 256 IP addresses (4 hosts up) scanned in 4.92 seconds`.

Answer: `4` (see 3.24). Subtract your own machine if the question asks for "other hosts".

Task 15: Find a MAC address

Scenario: Which IP on the local subnet belongs to a Xiaomi device?

```
sudo nmap -sn 192.168.31.0/24
```

Expect: `Nmap scan report for 192.168.31.138` with `MAC Address: B8:3B:CC:07:99:35 (Xiaomi Communications)`.

Answer: `192.168.31.138`. `netdiscover` (3.25) shows the same vendor column.

Task 16: Silent Windows host

Scenario: `nmap 10.4.19.218` says "Host seems down". Find the open ports.

```
nmap -Pn 10.4.19.218
```

Expect: ports 80, 135, 139, 445, 3389, 49154, 49155 open.

Answer: read the PORT table. RDP is `3389/tcp ms-wbt-server` (see 3.26).

Task 17: Fast scan with versions

Scenario: Quickly list services and versions on `10.10.10.5`.

```
nmap -Pn -F -sV 10.10.10.5
```

Expect:

```
22/tcp open ssh     OpenSSH 7.2p2 Ubuntu
80/tcp open http    Apache httpd 2.4.18
```

Answer: copy the VERSION column text. Then run `-p-` for ports `-F` can miss (see 3.27, 3.28).

Task 18: OS and page title

Scenario: What OS does `10.10.10.9` run, and what is the HTTP page title?

```
sudo nmap -Pn -O -sC -p 80,445 10.10.10.9
```

Expect: `|_http-title: Under Construction` and `OS details: Microsoft Windows Server 2012 R2`.

Answer: the `http-title` text and the OS details line (see 3.29).

Task 19: Version from page source

Scenario: What CMS version does `http://10.10.10.5` show, when `curl -I` only returns `Server: Apache`?

```
curl -s http://10.10.10.5 | grep -i "generator"
```

Expect: `<meta name="generator" content="WordPress 6.5.3" />`.

Answer: `WordPress 6.5.3` (see 3.30).

Task 20: Directory listing

Scenario: Find a browsable directory on `http://10.10.10.5`.

```
dirb http://10.10.10.5 -w | tee dirb.txt
curl http://10.10.10.5/files/
```

Expect: `==> DIRECTORY: http://10.10.10.5/files/` and a page titled `Index of /files`.

Answer: read the listed file names, then `curl` the file you need (see 3.31).

Task 21: Config backup

Scenario: Find the database user in a leftover WordPress config copy.

```
for ext in .bak .old .orig .backup '~' .save; do
  code=$(curl -s -o /dev/null -w "%{http_code}" \
    http://10.10.10.5/wp-config.php$ext)
  echo "$code -> wp-config.php$ext"
done
curl -s http://10.10.10.5/wp-config.php.bak | grep DB_USER
```

Expect: `200 -> wp-config.php.bak` and `define( 'DB_USER', 'wpuser' );`.

Answer: `wpuser` (see 3.32).

Task 22: Mirror and search

Scenario: A file linked only from an inner page contains the word "password".

```
httrack http://10.10.10.5 -O /tmp/mirror
grep -rni "password" /tmp/mirror/
```

Expect: `/tmp/mirror/10.10.10.5/old/notes.txt:3:password for ftp is ...`.

Answer: the path, line number and text of the match (see 3.13).

6.2 Exam-style questions

Q1 (multiple choice). Which activity is passive reconnaissance?

- A. `nmap -sn 10.10.10.0/24`
- B. `whois example.com`
- C. `dirb http://example.com`
- D. `dig axfr @ns1.example.com example.com`

Q2 (multiple choice). A host does not answer ping and Nmap reports "Host seems down" for a Windows target. Which option should you add?

- A. `-sn`
- B. `-F`
- C. `-Pn`
- D. `-O`

Q3 (multiple choice). Which DNS record type maps an IP address back to a hostname?

- A. MX
- B. PTR

- C. SRV
- D. CNAME

Q4 (fill in). To restrict a Google search to one domain, use the operator `_____`.

Q5 (fill in). The Nmap flag that probes open ports for software names and versions is `_____`.

Q6 (find the value). An Nmap default scan prints `Not shown: 993 filtered ports` and lists 7 ports as open. How many ports did Nmap scan in total?

Q7 (find the value). From this zone line, which number is the SOA serial?

```
zonetransfer.me. 7200 IN SOA nsztm1.digi.ninja. robin.digi.ninja.
    2019100801 172800 900 1209600 3600
```

Q8 (multiple choice). `dig axfr ns1.example.com example.com` returns "Transfer failed." twice. What is the most likely cause?

- A. The domain does not exist
- B. The `@` is missing before the server name
- C. DNSSEC is enabled
- D. The server is offline

Q9 (multiple choice). In which order should recon steps usually run?

- A. Active, passive, scope, document
- B. Scope, passive, active, document
- C. Passive, scope, document, active
- D. Document, scope, active, passive

Q10 (find the value). The command is `curl -s http://target.ine.local | grep -i "generator"` and the line is `<meta name="generator" content="WordPress 6.5.3 - ..." />`. What CMS version does the site run?

6.3 Answer key

Q1: B. `whois` queries a registry, not the target. The others send traffic to the target or its DNS (3.3, 3.10).

Q2: C. `-Pn` skips host discovery and scans ports anyway (3.26).

Q3: B. PTR is the reverse record (3.19).

Q4: `site:` (3.17).

Q5: `-sV` (3.28).

Q6: 1000. Default Nmap scans the 1000 most common TCP ports: 993 filtered plus 7 open (3.26).

Q7: `2019100801`. The serial is the first number after the admin mailbox (3.19).

Q8: B. Without `@`, dig treats the server name as a second name to query, so two failed lines appear (3.21).

Q9: B. Scope, passive, active, document (3.5).

Q10: 6.5.3. The version sits in the `content` value of the generator tag (3.30).

7. Recap Card and Flashcards

7.1 Recap Card

One line per command. The number at the end is the block where it is fully explained.

```
host hackersploit.org                                # 3.6
host -t mx example.com                               # 3.6
curl http://target.ine.local/robots.txt              # 3.7
curl -s http://target.ine.local/sitemap.xml          # 3.7
whatweb hackersploit.org                             # 3.9
whatweb -a 3 -v http://10.10.10.5                    # 3.9
whois hackersploit.org                               # 3.10
httrack http://target.ine.local -O /tmp/mirror       # 3.13
grep -r "flag\|FL@G\|FLAG" /tmp/mirror/              # 3.13
dnsrecon -d zonetransfer.me                          # 3.14
dnsrecon -d zonetransfer.me -t axfr                  # 3.14
wafw00f airtel.in                                    # 3.15
sublist3r -d hackersploit.org -e google,yahoo        # 3.16
theHarvester -d INE -b duckduckgo                    # 3.18
dig example.com MX +short                            # 3.20
dig TXT _dmarc.example.com +short                    # 3.20
dig -x 10.10.10.5                                    # 3.20
dig axfr @nsztm1.digi.ninja zonetransfer.me          # 3.21
host -l zonetransfer.me nsztm1.digi.ninja            # 3.21
dnsenum --enum zonetransfer.me                       # 3.22
dnsenum -f /usr/share/dnsenum/dns.txt example.com    # 3.22
fierce -dns zonetransfer.me                          # 3.23
fierce --domain zonetransfer.me                      # 3.23
sudo nmap -sn 192.168.31.0/24                        # 3.24
fping -a -g 10.10.10.0/24 2>/dev/null                # 3.24
sudo apt-get install netdiscover -y                  # 3.25
netdiscover -i eth0 -r 192.168.2.0/24                # 3.25
nmap -Pn 10.4.19.218                                 # 3.26
nmap -Pn -F 10.4.19.218                              # 3.27
nmap -Pn -p- 10.10.10.5                              # 3.27
nmap -Pn -F -sV 10.4.19.218                          # 3.28
nmap -Pn -F -sV -O -sC 10.4.19.218 -v                # 3.29
curl -s http://target.ine.local | grep -i generator # 3.30
dirb http://target.ine.local -w > dirb.txt           # 3.31
gobuster dir -u URL -w WORDLIST -x php,bak           # 3.31
curl http://target.ine.local/wp-config.php.bak       # 3.32
```

The backup-suffix loop is multi-line. Copy it from 3.32.

Google dorks (explained in 3.17):

```
site:ine.com inurl:forum
site:*.ine.com intitle:admin
intitle:"index of"
```

7.2 Flashcards

- Q: What is the first step of every pentest recon? | A: Define the target and confirm the scope (3.2, 3.5).

- Q: What does target scoping answer? | A: What am I allowed to collect information about (3.2).
- Q: Passive recon in one line? | A: Collecting information without directly engaging the target (3.3).
- Q: Active recon in one line? | A: Sending traffic to the target; needs authorization (3.3).
- Q: Which recon type comes first? | A: Passive (3.3, 3.5).
- Q: Name the four recon steps. | A: Define target, passive, active, document (3.5).
- Q: What does `host example.com` show? | A: A, AAAA and MX records (3.6).
- Q: What does Disallow in robots.txt reveal? | A: Paths the owner wants hidden from crawlers (3.7).
- Q: What does `whatweb` do? | A: Identifies web technologies on a site (3.9).
- Q: WhatWeb aggression level 1 vs 3? | A: 1 stealthy, one request; 3 aggressive, extra requests (3.9).
- Q: What does `whois` return? | A: Registrar, dates, contacts, name servers (3.10).
- Q: What does Netcraft SPF show? | A: Which servers may send mail for the domain (3.11).
- Q: What does a missing DMARC record mean? | A: No policy for failed SPF or DKIM; spoofing is easier (3.11).
- Q: What is HTTrack for? | A: Mirroring a site locally so you can grep it (3.13).
- Q: What does `dnsrecon -d` take? | A: The target domain (3.14).
- Q: What does wafw00f detect? | A: A Web Application Firewall and its vendor (3.15).
- Q: Is Sublist3r passive or active? | A: Passive, it uses search engines and OSINT (3.16).
- Q: What does `site:` do in Google? | A: Limits results to one site or domain (3.17).
- Q: What is the GHDB? | A: A database of Google dorks on exploit-db.com (3.17).
- Q: What does theHarvester `-b` choose? | A: The data source or sources (3.18).
- Q: Which DNS record gives an IPv6 address? | A: AAAA (3.19).
- Q: Which DNS record gives the mail server? | A: MX (3.19).
- Q: Which DNS record maps IP to name? | A: PTR (3.19).
- Q: What is NXDOMAIN? | A: The queried name does not exist (3.20).
- Q: What is a DNS zone transfer? | A: Copying the whole zone file from a name server (3.21).
- Q: Correct dig AXFR syntax? | A: dig axfr @server domain (3.21).
- Q: Why does a secure server answer "Transfer failed"? | A: It refuses AXFR requests from unknown clients (3.21).
- Q: What does `dnsenum -f` take? | A: A subdomain wordlist for brute force (3.22).
- Q: What is `fierce` used for? | A: Finding hostnames and scattered IP space through DNS (3.23).
- Q: What does `nmap -sn` do? | A: Host discovery only, no port scan (3.24).
- Q: What does `netdiscover` use to find hosts? | A: ARP, on the local network (3.25).

- Q: When do you use `nmap -Pn`? | A: When the host blocks ping and looks down (3.26).
- Q: What does `nmap -F` scan? | A: The top 100 ports (3.27).
- Q: What does `-sV` do? | A: Detects service names and versions (3.28).
- Q: What do `-O` and `-sC` do? | A: OS detection and default NSE scripts (3.29).
- Q: Why `curl -s URL | grep -i generator`? | A: Finds the CMS and version in the page body (3.30).
- Q: What does `dirb -w` do? | A: Does not stop on warning messages (3.31).
- Q: Why is `wp-config.php.bak` dangerous? | A: It is served as plain text and may hold DB credentials (3.32).
- Q: Why quote the tilde in the backup loop? | A: An unquoted ~ expands to the home directory (3.32).

8. Glossary

- **A record:** DNS record that maps a name to an IPv4 address.
- **AAAA record:** DNS record that maps a name to an IPv6 address.
- **Active reconnaissance:** Collecting information by sending traffic to the target.
- **ARP:** Protocol that maps an IP address to a MAC address on a local network.
- **ASN:** Autonomous System Number; an ID for a network run by one organization.
- **AXFR:** Full DNS zone transfer request.
- **Backup file:** A copy of a file (for example `.bak`) left on a server.
- **Banner:** Text a service sends that names its software and version.
- **Brute force:** Trying many guesses from a list until one works.
- **CDN:** Content Delivery Network; caching servers that sit in front of a site.
- **CIDR:** Notation like `/24` that sets the size of an IP range.
- **CMS:** Content Management System, such as WordPress.
- **CNAME record:** DNS alias that points one name to another.
- **Directory listing:** A server page that shows a folder's files because no index page exists.
- **DMARC:** Email policy that tells receivers what to do when SPF or DKIM fail.
- **DNS:** Domain Name System; maps names to IP addresses.
- **DNSSEC:** Signing of DNS data to prevent tampering.
- **Dork:** A search query that uses operators to find specific exposed content.
- **Filtered (Nmap):** No reply to the probe; a firewall is likely dropping packets.
- **Fingerprinting:** Identifying software or an OS from how it responds.
- **GHDB:** Google Hacking Database, a collection of dorks.
- **HINFO record:** DNS record with host CPU and OS text.
- **Host discovery:** Finding which IP addresses have live machines.
- **ICMP:** Protocol used by ping.
- **In scope:** Assets you are allowed to test.

- **MAC address:** Hardware address of a network card.
- **Mirror:** A full local copy of a website.
- **MX record:** DNS record that names the mail server for a domain.
- **Name server:** DNS server that holds records for a domain.
- **NS record:** DNS record that names a domain's name servers.
- **NSE:** Nmap Scripting Engine; runs scripts for extra checks.
- **NXDOMAIN:** DNS reply meaning the name does not exist.
- **OSINT:** Open Source Intelligence; information from public sources.
- **Out of scope:** Assets you must not test.
- **Passive reconnaissance:** Collecting information without touching the target.
- **PTR record:** DNS record that maps an IP address to a name.
- **RDP:** Remote Desktop Protocol, usually TCP port 3389.
- **Resolver:** DNS server that answers lookups on behalf of a client.
- **RFC1918:** Private IPv4 ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.
- **robots.txt:** File that tells crawlers which paths to avoid.
- **Scope:** The list of systems you are authorized to test.
- **SMB:** Server Message Block; Windows file sharing on TCP port 445.
- **SOA record:** DNS record with zone authority data and serial number.
- **SPF:** Email record that lists servers allowed to send mail for a domain.
- **SRV record:** DNS record that gives the host and port of a service.
- **Subdomain:** A name under a domain, such as `mail.example.com`.
- **TLD:** Top-level domain, such as .com or .org.
- **TTL:** Time to live; how long an answer can be cached.
- **TXT record:** DNS text record, used for SPF and verification.
- **WAF:** Web Application Firewall that filters attack-like web requests.
- **Web server:** Software that answers HTTP requests, such as Apache or IIS.
- **Wordlist:** A text file of words used for brute force.
- **XSS:** Cross-site scripting; running attacker script in a victim's browser.
- **Zone file:** The full set of DNS records for a domain.
- **Zone transfer:** Copying a DNS zone from one server to another.

9. Coverage Report

9.1 Page-to-section map

Notes page	Content	Guide section
1	Information gathering, passive and active	3.1
1	Target scoping, target types, in and out of scope	3.2
2	Passive and active recon slides	3.3
2, 4	What to collect, what we are looking for	3.4
3	Recon flow, four-step strategy, mistakes, takeaways	3.5
4	host command	3.6
4, 10	robots.txt, sitemap.xml, CTF Flag 1	3.7
4	BuiltWith, Wappalyzer	3.8
5	WhatWeb	3.9
5	whois	3.10
5	Netcraft, SPF, DMARC, trackers	3.11
5	DNSDumpster	3.12
5, 12	HTTrack, CTF Flag 5	3.13
5	dnsrecon	3.14
5	wafw00f	3.15
6	Sublist3r	3.16
6, 7	Google dorks, GHDB	3.17
7	theHarvester	3.18
8	DNS, DNS records	3.19
8	DNS interrogation, dig	3.20
8	Zone transfer, dig axfr	3.21
8	dnsenum	3.22
8	fierce	3.23
9	nmap -sn	3.24
9	netdiscover	3.25
9	nmap -Pn	3.26
9	nmap -F	3.27
9	nmap -sV	3.28
9	nmap -O -sC -v	3.29
10	CTF Flag 2 (generator tag)	3.30
11	CTF Flag 3 (dirb)	3.31
11, 12	CTF Flag 4 (backup files)	3.32

Notes page	Content	Guide section
9, 13	CTF header and summary	3.33

9.2 Source limits and [UNCLEAR] items

The notes reached this guide as a page-by-page text transcription of the screenshots. The original images were not available, so nothing was checked by eye against the pictures. Anything below that was missing from the transcription is marked.

- [UNCLEAR on page 4] Wappalyzer panel contents were described but not transcribed. Section 3.8 uses an illustrative panel.
- [UNCLEAR on page 5] The result of `whatweb hackersploit.org` was not shown. Output in 3.9 is illustrative.
- [UNCLEAR on page 5] DNSDumpster result rows were not transcribed. Section 3.12 lists only the visible headings and hosting names.
- [UNCLEAR on page 6] Output of `sublist3r -d hackersploit.org -e google,yahoo` was not shown. Output in 3.16 is illustrative.
- [UNCLEAR on page 7] theHarvester result sections were not shown, only banners and warnings. Result sample in 3.18 is illustrative.
- [UNCLEAR on page 8] The `dnsenum` tail has a cropped left edge (`...e.`). Best reading: the zone name `zonetransfer.me.` (uncertain).
- [UNCLEAR on page 8] The `fierce -h` text is cut off after the `-connect` option.
- [UNCLEAR on page 9] Outputs of `nmap -Pn -F -sV` and `nmap -Pn -F -sV -O -sC -v` were not shown. Outputs in 3.28 and 3.29 are illustrative.
- [UNCLEAR on page 9] Why `netdiscover` was still "command not found" after install is not shown. The reason in 3.25 is a likely cause (verify).

Conflicts and corrections:

- Page 2: slide titled "passive" lists active data. Corrected in 3.3.
- Page 8: `dig aniketpandey.website all` treats `all` as a query name. Explained in 3.20.
- Page 8: `dig axfr` without `@` in the first example. Explained in 3.21.
- Page 12: bare `~` in the loop is expanded by bash. Quoted version in 3.32.
- Page 7: `-d INE` is a company name, not a domain. Noted in 3.18.
- Page 3: the diagram text was garbled. Rebuilt in 3.5.
- No conflict was found with the exam facts you supplied.

9.3 Items to double check

- WhatWeb aggression level 4 and version of WhatWeb on your Kali (3.9).
- dnsrecon `-t` type names and brute-force wordlist path (3.14).
- Sublist3r extra options `-p` and `-v`, and CAPTCHA behavior of search engines (3.16).
- theHarvester source list in your version, and where API keys are stored (3.18).

- Modern `fierce` syntax (`--domain`) versus the old `-dns` in your notes (3.23).
- Whether `dig axfr` without `@` behaves as described on your dig version (3.21).
- Default Nmap host-discovery probes used by `-sn` on remote networks (3.24).
- Cause of `netdiscover: command not found` after install (3.25).
- `dirb` default wordlist path and Gobuster flags (3.31).
- HTTrack `-s0` robots option and WPScan `--enumerate cb` (3.13, 3.32).
- Whether BuiltWith works inside an offline exam lab (3.8).

9.4 Coverage confirmation

Every item of the page inventory maps to a section in 9.1. Each item has its theory, commands, flags, sample output, worked example and lab angle in a single concept block. The five CTF flags are covered in 3.7, 3.30, 3.31, 3.32 and 3.13, with the overview in 3.33. The 76-character code limit means long output lines (WHOIS, dig, the generator tag) are wrapped; their values are unchanged.